OpenText™ Service Manager

Develop

Version 9.82

PDF Generated on October 12, 2026

© Copyright 2026 Open Text

Table of Contents

Develop 5

1 Tailoring 5

1.1 Environment configuration 5

1.1.1 Tools to use for promoting customization 6

1.1.1.1 Development auditing 7

1.1.1.2 Unload script utility 11

1.1.1.3 Differential Upgrade utility 11

1.1.1.3.1 Patch records 12

1.1.1.3.2 Signature records 12

1.1.1.3.3 Checklist: Differential upgrade steps 12

1.1.1.4 Revision control 15

1.1.2 Best practices for promoting customizations 17

1.1.3 Promote customizations from development to production 18

1.2 Data management 19

1.2.1 Fields and Keys tab 19

1.2.1.1 Add a field to a table 25

1.2.1.2 Edit a field in a table 26

1.2.1.3 Edit a record key 27

1.2.1.4 Update the size of a field 28

1.2.2 Cascade Updates 28

1.2.3 Data Policy 30

1.2.3.1 Access Data Policy 30

1.2.3.2 Data Policy expressions 31

1.2.3.3 Usage Type column in Data Policy 31

1.2.3.4 Data Policy and the object record 32

1.2.3.5 Creating revisions 32

1.2.3.5.1 Example: Create and manage a revision 33

1.2.3.5.2 Create a Data Policy revision 33

1.2.3.6 Revert to a previous revision 34

1.2.3.7 Purge revisions 34

1.2.3.8 Data Policy and encryption 34

1.2.3.9 Change the encryption key value 35

1.2.3.10 Create an alias for your custom CI display name field 35

1.2.3.11 Define auto complete table columns 36

1.2.3.12 Specify a referenced table for a field 37

1.2.3.13 Specify a display field for a referenced table 38

1.2.4 Data validation 39

1.2.4.1 Accessing the validity file 39

1.2.4.2 Deleting validity record components 40

1.2.4.3 Invalid application names 40

1.2.4.4 Invalid files and form names 41

1.2.4.5 Invalid field names 41

1.2.4.6 Printing validity definitions 41
1.2.4.7 Invoking validity table processing 42
1.2.4.8 Look-up processing 42
1.2.4.8.1 Format Control processes 42
1.2.4.8.2 Displayoptions 43
1.2.4.8.3 Display value summary details 44
1.2.4.8.4 Display range summary details 45
1.2.4.8.5 Create validity table definitions 45
1.2.4.8.6 Add field level definitions 46
1.2.4.8.7 Add value definitions 47
1.2.4.8.8 Add range definitions 47
1.2.4.8.9 Validate the validity definitions 48
1.2.4.8.10 Delete an entire validity record 49
1.2.4.8.11 Delete a single value or range definition 50
1.2.4.8.12 Delete value or range definitions from a table 50
1.2.4.8.13 Print a detailed report of an entire validity record 51
1.2.4.8.14 Validate fields during record processing 52
1.2.4.8.15 Create a validity lookup option in Format Control 54
1.2.4.8.16 Call validate.fields from Format Control 55
1.2.4.9 Validity validation rules 56
1.2.5 Global lists 57
1.2.6 Record tag localization utility 62
1.2.6.1 Localization process 62
1.2.6.1.1 Enable a table for tag localization 62
1.2.6.1.2 Access and localize message records 63
1.2.6.1.3 Create localized global lists 63
1.2.6.1.4 Invoking the localization utility 64
1.2.6.2 Codes 65
1.2.6.2.1 Defining a set of values for a codes-based global list 65
1.2.6.2.2 Accessing and localizing message records for a codes-based global list 66
1.2.6.2.3 Creating localized codes-based global lists 66
1.2.6.2.4 Associate a field with a codes-based global list 67
1.2.6.3 Message maintenance with record tag localization 67
1.2.7 Help systems 68
1.2.7.1 Online Help system 68
1.2.7.2 Field help editor 68
1.2.7.2.1 Creating field help 69
1.2.7.2.2 Preparing to create field help 69
1.2.7.2.3 Access the help table 71
1.2.7.2.4 Check the fields that a form contains 71
1.2.7.2.5 Check the fields that a table contains 72
1.2.7.2.6 Check whether help exists for a field 72
1.2.7.2.7 Detect the help record to display 73
1.2.7.2.8 Add or edit help records 73
1.2.7.2.9 Review field help records 74

1.2.8 Sequential number setup 74
1.2.8.1 Sequential number file 75
1.2.8.2 Access the sequential number file 75
1.2.8.3 Create a simple number counter 75
1.2.8.4 Use decrement in sequential numbers 76
1.2.8.5 Use prefix and suffix in sequential numbers 76
1.2.8.6 Update a sequential number record 77
1.2.8.7 Delete a sequential number record 77

Develop

Tailoring

Programming

Document Engine

Web Services

SMIS

Tailoring best practices

Process Designer tailoring best practices

Wizards

Processes and best practices

Database reports

Service Manager third-party components

1. Tailoring

Service Manager Tailoring has information for System Administrators and implementers who install and configure Service Manager. Use Service Manager Tailoring to make further changes to support site-specific requirements, including special field validation, new or modified forms design, expanded or varied workflow, and automatic notifications.

Tailoring is any change to standard functionality without changing actual code. For example, you can:

- Change the look and operation of forms.
- Change default values for objects on forms that Service Manager uses for field validation.
- Create macros, scripts, and stored queries.
- Make changes to record definitions.

Important

Important Service Manager doesn't support excessive tailoring such as changing the user access of a module. For example, the Incident Management module is intended for power users only; if you tailor your system to make this module accessible through the employee self-service (ESS) interface, unpredictable results could occur to your system. For more information about the Windows client and different web client views, as well as their target user groups, see Service Manager Web tier.

Related topics

Environment configuration
Data management
Form creation
Links
Display functions
Advanced functions
Integrations
Service Manager Programming

1.1. Environment configuration

A complete Service Manager environment consists of at least three different instances of Service Manager:

- A development instance, which is used to customize Service Manager to the exact needs of the customer

- A test instance, which is used to test these changes in a copy of the production system before promoting the customizations into production
- The production instance itself

These topics focus on environment configuration options within the system. The main utilities that enable environment configuration are:

- Development auditing — tracks changes to records during the development phase of implementation.
- Differential Upgrade — moves changes from a development environment into a test or production environment.
- Document Engine — defines processing rules for files.
- Revision control — reverts to a previous version of a file or format.
- Wizard creation tool — simplifies repetitive tasks.

Related topics

Tailoring
Tools to use for promoting customizations
Development auditing
Revision control
Differential Upgrade utility
Unload script utility
Document Engine
Wizards

1.1.1. Tools to use for promoting customization

Service Manager offers the system developer different tools to make the transitions from development to test, and from test to production, easier.

- Development auditing
- Unload script utility
- Differential Upgrade utility
- Revision control
- Change Management

Comparison of the tools to use for promoting customization

Each method of promoting customization has advantages and disadvantages. The following table compares what each method is capable of providing.

Note: Manual processing involves writing down each change as it is being made, and then unloading records one-by-one manually via the Service Manager Database Manager. Manually unloading each record takes a lot of time and is prone to errors.

Ability	Development auditing	Unload script utility	Differential Upgrade Utility	Manual Processing
Can monitor all files?	No. Hard-coded list of tailoring files	Yes	Yes	Yes
Possible to specify a detailed query?	By date only	Yes	Yes	Yes
Can select and deselect records to unload?	Yes, by removing records from the devaudit file	No	Yes	No
Can do dbdict field merge?	No	No	Yes on fields, but not on keys	Yes, by changing the dbdict load option
Are changes to the record included in unload?	All	Latest	Latest	Latest
Performance Ranking, includes creating and applying patch	Poor	Good	Excellent	Fair
Ranking by ease of use	Good	Excellent	Fair	Poor
Ranking by ease to set up	Excellent	Good	Fair	Poor

Related topics

Development auditing
Unload script utility
Differential Upgrade utility
Revision control

1.1.1.1. Development auditing

Applies to User Roles: System Administrator

The development auditing utility (devaudit) tracks changes made to Service Manager records during the development phase of Service Manager implementation. Whether you make a few changes or extensively customize your system, it is critical to have a record of your changes to ensure loading of the correct version when you move to production. With it, you can create an unload of your changes, and load them into another system.



Note

Note The Differential Upgrade utility is a more powerful and flexible tool for this purpose. We recommend that you use it instead of the development auditing.

Though the auditing utility helps you find modified records, strongly recommends that you record each change more extensively either using tools outside of Service Manager or using the Revision control feature.

The development auditing utility tracks changes to the following files:

- application
- code
- datadict

- devaudit
- displayevent
- displayoption
- displayscreen
- enclapplication
- eventfilter
- eventmap
- eventregister
- format
- formatctrl
- joindef
- link
- menu
- Object
- Process
- querystored
- scmessage
- screlconfig
- scripts
- States
- trigger
- validity
- wizard

Access the development auditing utility[edit]

To access the development auditing utility:

1. Click **Tailoring** > **Audit**.
2. Select a function from the Audit menu.
    - **Audit Log**: The repository for data gathered during the audit. It displays the old and new values of input field data, as well as who made the revisions and when.
    - **Audit Specifications**: Provides instructions on how and when to perform an audit. Use it to define files and fields that you want the audit application to monitor.
    - **Purge Audit History**: After the development phase is completed and all related changes have been transferred into the production system, you can purge the development audit data. Ensure you carefully review the audit records in the Audit History record list before filling in the date to purge files, as you can't restore purged files.
    - **Turn Auditing On/Off**: Enable or disable file auditing. If auditing is on, the system creates a separate audit record each time you make a change to a record and saves it to the database. Service Manager contains a default set of audit records to enable you to track development changes.
    - **Unload an Audit Delta**: Unload a change you made during the development phase. It is critical that you check the audit files for the correct date of this delta. You need the date shown in the Audit History form to enter into the Unload form. When the Audit Unload form opens, the current date and time automatically fill in the **Unload delta since what date?** field. Type the beginning date for the unload that's found in the **View Audit History** record list.
    - **View Audit History**: Search for a particular audit record or view a record list of all audits stored in the database from the Audit History form. It is important to look at this file before purging audit records or unloading a development change to production.

Turn the auditing on/off[edit]

This function enables you to enable or disable file auditing. If auditing is on, the system creates a separate audit record each time you make a change to a record and save it to the database. Service Manager contains a default set of audit records to enable you to track development changes.

1. Click **Tailoring** > **Audit** > **Turn Auditing On/Off**.
2. Update one or more of the following functions:
    - **Do you want to audit development changes?**: Enables/disables auditing of development changes. When selected, development auditing tracks the changes you make. You can put those changes into an unload file and load them into another system.
    - **Do you want to keep backups of Changes?**: Enables/disables backups of development changes. When selected, development auditing creates a copy of the record you changed in the same table. The name of the backup record is prefaced with AUDIT and the number of the audit record and key. Restores the originals by renaming them to the original file name.
    - **Do you want to create auditdelete records on unload?**: Enables/disables creating auditdelete records on unload. When selected, development auditing tracks your deletes, and the unload file makes the same deletes when you load it into another system.
3. Click **Save** and **OK**.

Best practices

Auditing should always be disabled on a production or test system, and enabled for development systems (when auditing is used to promote from development to production).

When selecting auditing settings for a development system:

- Select **Do you want to audit development changes?**
- Uncheck **Do you want to keep backups of changes?**
- Uncheck **Do you want to create auditdelete records on unload?**

View the audit history[edit]

Search for a particular audit record or view a record list of all audits stored in the database from the Audit History form. It is important to look at this file before purging audit records or unloading a development change to production.

If some of the records should not be added to the auditing delta unload, you can either change the search criteria of these records (such as changing the date to an earlier date) or delete the record altogether if it should never be part of the delta and doesn't need to be documented as changed.

Note: Be aware that the same record may be included in the devaudit file several times, since each change to a record is noted in devaudit.

1. Click **Tailoring** > **Audit** > **View Audit History**.
   A blank Audit History record opens.
2. Open an existing Audit record using one of the following procedures:
    - Enter any information you have from the record and click **Search**.
    - Click **Search** or press Enter. If more than one record matches the search criteria, the result screen is split with a record list at the top and the first record in the list open below.

   Note: There is an entry for each time you add or update a form or file.
3. Click the entry you want to view. Information for this record is entered into the Audit History form.
4. Click **Delete** to remove the current record from the audit list.

The records have the following structure:

- Audit ID: 1
- Filename: format
- Keys: IM.update.incident
- Event Type: update
- Date: 01/14/10 12:31:29

- Operator: falcon
- Last Record: true

Unload an audit delta[edit]

To specify what the system unloads, you must first view the audit history. Remove records from the development audit list by selecting them and then deleting them.

Important: Before unloading a change you made during the development phase, it is critical that you check the audit files for the correct date of this delta. You need the date shown in the Audit History form to enter into the Unload form.

To unload the audit delta:

1. Click **Tailoring** > **Audit** > **Unload an Audit Delta**.
   The Audit Unload form opens, and the current date and time automatically fill in the **Unload delta since what date?** field.
2. Type the beginning date for the unload that's found in the **View Audit History** record list.
3. Type the path and file name that you want to send the unloaded data in the **Send Data to Which File?** field.
4. Type the path and file name of the log for the audit unload process in the **Enter Log Name Here** field.
5. Click **Proceed**. The Development Auditor menu opens with a message in the status bar indicating how many records were unloaded.

The unload you create can be loaded into the test system, and eventually into the production system.

Note: You can view the contents of an unload file in Database Manager before loading it onto the new system.

To view the contents of an unload file:

1. Open Database Manager.
2. Select **Import/Load** from the More Actions menu.
3. Select the file you want to load.
4. Click **View Contents**.

Purge audit records[edit]

After the development phase is completed and all related changes have been transferred into the production system, you can purge the development audit data. You are prompted to specify the start date for records to be purged. After the data is purged, it can't be restored (unless an unload of the devaudit file was performed beforehand).

To purge audit records:

1. Click **Tailoring** > **Audit** > **Purge Audit History**.
   The Audit Purge form opens with the current date and time filled in.
2. Replace the date displayed with the beginning date for the range of records you want to delete.
   Warning: Ensure that you carefully review the audit records in the Audit History record list before filling in this date. You can't restore purged files.
3. Click **Proceed**. A confirmation prompt opens informing you how many records have been purged.
4. Click **OK** to complete the process and return to the Development Auditor menu.

Contents

- 1 Access the development auditing utility
- 2 Turn the auditing on/off
- 3 View the audit history
- 4 Unload an audit delta
- 5 Purge audit records

1.1.1.2. Unload script utility

The unload script utility enables system administrators to create Service Manager unload files automatically. The unload script utility enhances the standard unload creation process in many ways. With unload scripts, you can:

- Save records from multiple tables into a single unload file.
- Specify a query for each source table that filters the records added to the unload file.
- Purge records during the unload process.
- Specify which formats to protect during a purge process.
- Add related records from the data map file to the unload file.

The unload script utility is available in Tailoring. By default, Service Manager includes a collection of unload scripts that you can use for common unload tasks. You can also use the default unload scripts as templates to create your own customized unload scripts.

To create and use unload scripts effectively, you must be familiar with the Service Manager Database Manager. You can create a query on any field in the file, but querying keyed fields improves system performance and response time. You can create record queries using Structured Query Language (SQL) syntax.

You can use the Purge/Archive utility to unload and purge records from one file; if you use datamaps, you can use Purge/Archive to unload and purge related records as well.

After you load a .unl file to import data into Service Manager, you must stop and then restart the Service Manager server. This ensures that the table changes are initialized.

Note: The Unload Script utility unloads tables using the binary unload file format.

Related topics

Tools to use for promoting customizations
Data maps
Environment configuration
Exporting records
Importing records
Purging and archiving records
Using data maps to purge and archive records
Add an unload script record
Protect forms from the unload script utility
Purge records with the unload script utility
Run an unload script
Validate an unload script record

1.1.1.3. Differential Upgrade utility

The Service Manager Differential Upgrade utility enables a developer to compare the specified files of two Service Manager systems to create a single unload file. The resulting unload file contains all the necessary records to make the files identical between the two systems.

The Differential Upgrade utility simplifies the way you move changes from a development environment into a test or production environment. You can also use this utility to move files between any two systems, such as:

- Development and test
- Unit testing and acceptance testing
- Two different development systems

Important: We recommend that you understand how to export files and create different types of unload files before you use the Differential Upgrade utility.

The Differential Upgrade utility, just like the upgrade utility that upgrades applications from one version to another, relies on patch records and signatures. For more information on how to use the Differential Upgrade utility, see the Service Manager online Help topics under "Application Development - Differential Upgrade utility."

Related topics

Environment configuration
Tools to use for promoting customizations
Patch records
Signature records
Checklist: Differential upgrade steps
Load an unload file

1.1.1.3.1. Patch records

A patch record specifies which files the Differential Upgrade utility should compare, and creates a query that limits the comparison to certain records within files. There must be a patch record for each system in the comparison.

Note: The query should limit the number of records to upgrade. The more files in the patch record, the longer the Differential Upgrade process takes.

In a Differential upgrade comparison, you must have one patch record for each system to be compared. The patch record must point to identical files in each target system and generate the same queries. You can either create it once on each system; or create it on one system, unload it, and load it into the other system. You may prefer to create customized patch records before you start the Differential Upgrade process.

Note: Be sure to include all files that were modified during customizations that you promote to production.

One method of limiting the number of records included in the patch record is to use the `sysmodtime` field that many dbdicts contain. Service Manager updates this field automatically with each update of the record. If the file doesn't contain a `sysmodtime` field, you can either add it before starting the customization work on the development system, or use another limiting field such as `update.time`.

In the Patch record dialog box, you can specify that the Differential upgrade should perform Add only processing on a specific file. If you choose this option, Service Manager adds only new records to the Differential upgrade unload file and ignores changes to existing records in the development system.

Related topics

Environment configuration
Tools to use for promoting customizations
Differential Upgrade utility
Signature records
Checklist: Differential upgrade steps
Load an unload file

1.1.1.3.2. Signature records

A signature for an Service Manager record is a numerical representation of the record. Any change to the contents of the record causes the signature of that record to change, based on the definitions in the `signaturemake` file.

You must create signature records for any record that you compare in the Differential Upgrade process. The `signatures` file contains signature records.

Important: Never change a record in the `signaturemake` file.

Related topics

Environment configuration
Tools to use for promoting customization
Differential Upgrade utility
Patch records
Checklist: Differential upgrade steps
Load an unload file

1.1.1.3.3. Checklist: Differential upgrade steps

Applying a Differential upgrade from a development system to a production system requires these steps:

1. Create separate Patch records for the development and production systems. The patch records must reference the same files on each system.
2. Create Signature records for the production (target) system.
3. Move Signature records to the development system.
4. Create the Differential upgrade from the development (source) system. You can create an external unload file or an internal file.
5. Customize the Differential upgrade unload files. This is optional if you created an internal file.
6. Create a differential upgrade unload from internal files.
7. Load the Differential upgrade unload into the production (target) system.

Create or update a Patch record

Applies to User Roles: System Administrator

To create or update a Patch record:

1. Click **Tailoring** > **Differential Upgrade** > **Patch Records**.
2. Type the name of the Patch record in the **Patch Name** text box.
3. Type a query (or false) in the **Application Clusters** text box. For example: `application ="app_name"`
4. Type a query (or false) in the **Format Records** text box. For example: `name ="format_name"`
5. Type the names of files to be included in the Patch record in the Dbdict fields.
6. Click the **Add** checkbox if you want to Add new records only. Otherwise, the Differential Upgrade utility processes both updated and new records.
7. Type true, false, or an advanced query in the Query field for each file. True includes all records from the file; false doesn't return any records. An advanced query enables you to customize the results.
8. For a new Patch record, click **Add**.
9. To update a Patch record, click **Save**.
10. Click **OK**.

There must be a Patch record for each system in the comparison. Each patch record must point to the same files and records. You can create it once on each system, or create it on one system, unload it, and load it into the other system.

Create Signature records on the production system

Applies to User Roles: System Administrator

To create Signature records on the production system:

1. Click **Tailoring** > **Differential Upgrade** > **Differential Upgrade Wizard**.
2. Click **Build Signatures**.
3. Click **Next**.
4. Type the name of the system in the **System Name** text box. Remember the name because Service Manager uses it in later steps.
5. Click the drop-down list to choose an existing Patch record for the named system. Remember that there must be a Patch record for each system to be compared, and each Patch record must point to the same files and records.
6. Browse to locate or define the **Export Filename**. This is an external file that will store the signature records.
7. Optionally, click **Signature RAD components for delete processing** to delete all related RAD panel components.
8. Click **Run in background** if you want background processing. If you run in background, Service Manager creates a schedule record and creates the unload when the schedule record runs. If you choose Run in background, do the following:
   • Click **Repeat Daily** to create signature records daily. This step is optional.
   • Click the drop-down list to choose the **Date/Time to Run** schedule for background processing.
9. Click **Next**.

If the process runs in background, the unload file will reside on the server. If the process runs in foreground, the unload file location depends on whether you enable the Client Side Load/Unload option on the client. Processes running in

foreground post progress messages in Active Notes, if that option is enabled.

Move Signature records to the development system

Follow these steps:

1. Ensure the signatures file on the development system is empty.
2. Load the signatures file on the development system using the standard Service Manager Import/Load utility.

If you choose background processing, Service Manager moves the signature records automatically.

Create the Differential upgrade from the development system

Applies to User Roles: System Administrator

To create the differential upgrade from the development system:

1. Click **Tailoring** > **Differential Upgrade** > **Differential Upgrade Wizard**.
2. Click **Create Differential Upgrade**.
3. Click **Next**.
4. Double-click the correct Patch record from the list of Patch records. This should be the same Patch record specified when you created the production Signature records.
5. Type a descriptive name for the system you are building in **Current Version**.
6. Type the descriptive name for the system you are upgrading in **Compare to Version**. This should be the System Name you specified when you built the Signature records.
7. Optionally, select **Perform Delete Processing** to delete records in the target system.
8. Decide whether you want to create an external or internal unload file. Depending on your choice, complete Step 9 or Step 10.
9. To create an external unload file and a text file listing all records included in the unload file, click **Create Unload** and then click **Next**.
   Click **Browse** to locate a file, or define an **Export Filename** for the unload file containing the Differential Upgrade.
   Click **Browse** to locate a file, or define a **Log Filename**.
   Click **Run in background** if you want background processing. Service Manager creates a schedule record and creates the unload when the schedule record runs. If you choose Run in background, do the following:
   a. Click **Repeat Daily** to create signature records daily. This is an optional step.
   b. Click the drop-down list to choose the **Date/Time to Run** schedule for background processing.
   c. Click **Browse** to locate the **Path to Signatures unload** where the signature file resides. For background processing, the signatures unload file must be on the Service Manager server.
   Click **Next**. When the Differential upgrade is complete, the file named in Export Filename contains the information that aligns the production system with the development system. There will be a record for each difference between the development and production systems.
10. To add a record to the `diffupg` file for each record included in the upgrade, choose **Create Internal File** and then click **Next**.
    Click **Run in background** if you want background processing. If you run in background, Service Manager creates a schedule record and creates the unload when the schedule record runs. If you choose Run in background, do the following:
    a. Click **Repeat Daily** to create signature records daily. This is an optional step.
    b. Click the drop-down list to choose the **Date/Time to Run** schedule for background processing.
    c. Click **Browse** to locate the **Path to Signatures unload** where the signature file resides. For background processing, the signatures file must be on the Service Manager server.
    Click **Next**. When the Differential upgrade is complete, there is one record for each record included in the upgrade.

Customize the Differential upgrade unload file

Applies to User Roles: System Administrator

If you create an internal file, you can customize the unload file before processing begins. You can page through the `diffupg` file to examine each record, and specify whether it is to be included in the unload by selecting the Unload checkbox.

To customize the Differential upgrade unload file:

1. Ensure that you complete the steps to run the Differential upgrade wizard and create an internal file.
2. Click **Tailoring** > **Differential Upgrade** > **Differential Upgrade Records**.
3. Click **Search**.
4. Choose a record from the list of Differential upgrade records. Service Manager displays the record in the Differential Upgrade Results form.
5. Select or clear the **Unload?** checkbox to include or exclude the record from the Differential upgrade process.
6. Click **Next**.
7. Repeat these steps until you page through all of the records in the file.

Note: Click **Go To Record** to view the complete record.

Create a differential upgrade unload from an internal file

Applies to User Roles: System Administrator

To create a baseline revision:

1. Click **Tailoring** > **Differential Upgrade** > **Create Differential Unload**.
2. Type an **Export Filename**.
3. Type the **System Name**. You are creating differential upgrade unload records for only this system. This is the name of the system listed in the differential upgrade records.
4. Click **Next** to create the unload.

Load the Differential upgrade into the production system

Use the standard Service Manager import/load utilities to load the Differential upgrade unload file into your production system.

1.1.1.4. Revision control

Revision control provides developers and administrators with a means of reverting to a previous version of a file or form. If during the process of creating or modifying forms you find an error, revision control returns a working version of your file or form.

Revision control allows a developer to:

- Create a snapshot of a record
- Add SCR information and comments to the snapshot
- Replace the current version of the record with a working version of the record at any time.



Note

Note Every revision takes up as much disk space as the original record plus a few bytes for comments.

Use revision control in conjunction with development auditing to track, record, and save changes to your system. The development auditing utility provides a record of the changes to ensure that you load the correct version when you move to production, whereas revision control documents these changes and enables you to create working snapshots.

Service Manager handles revisions as part of the Document Engine. Revisions are available in all utilities that use the Document Engine as base code, including Database Manager, Format Control, Link Editor, Forms Designer, the RAD Editor, and others.

The system stores revisions in a separate file whose name is specified in either the object record for the file or in the datadict record. The system creates this revision file via an option on the Data Policy or the object screen. Administrators specify the maximum number of revisions to store for each record in a file. If you don't specify a number, an unlimited number is stored.

Administrators need to determine in advance in which files to track revisions and then do a minor setup to establish them. Administrators also need to purge revisions prior to migrating to a production system.

Create revisions

The revisions option for Data Policy creates a revision database dictionary record, whose name is specified on the Engine Specifications tab. You can create revisions either for an entire set of records or for a single form or record. You must be at the start panel of the application you are working with to create baseline revisions for an entire set of records. For example, you create a baseline revision of all the forms in the Forms Designer from the Forms Designer main menu.

When you create a revision, you create a copy of the record only, not the associated database dictionary record. As a result, any time a field is added to a database dictionary record, you add it to the revision file database dictionary record.

Create a baseline revision

Applies to User Roles: System Administrator

To create a baseline revision:

1. Open a search screen for a form or file that supports revisions. For example, the Forms Designer application.
2. Click **More** of the More Actions icon.
3. Click **Revision**.
4. Click **Create Baseline Revision**. A dialog box opens.
5. Enter the SCR information or click **Ok** to begin.

The system creates a copy of all the records in the application, in this case every form in the Forms Design application.

Create a single revision

Applies to User Roles: System Administrator

To create a single revision:

1. Open a form or file that supports revisions. For example, a RAD application in the RAD editor.
2. Click **More** of the More Actions icon.
3. Click **Create Revision**. The revision tracking panel opens.
4. Type in information relating to the revision. Make comments to help others understand the reason for the revision.
5. Click **Save**.

A copy of the revision saves and you return to the main panel.

Revert to a previous revision

Applies to User Roles: System Administrator

To revert to a previous revision:

1. Open the form you want to use a previous version of.
2. Click **More** or the More Actions icon.
3. Click **Revision**.
4. Click **Revisions > Find Revisions**.
5. Click the revision you want to restore.
6. Click **Revert to this Revision** from the More Actions menu. The system prompts you as to whether you want to save the current version of the record as a revision. It is recommended you do.

Search for revisions

Applies to User Roles: System Administrator

To access the wizard creation tool:

1. Open an application that supports revisions. For example, the RAD editor or the Forms Designer.
2. Click **More** or the More Actions icon.
3. Click **Revision > Find Revision**.
   You can search for revisions using one or more of up to five of the following search criteria:
   - syslanguage — Search for revisions by language indicator. For example, en for English.
   - name — Search for revisions by name. For example, cc.incquick.
   - Revision Date — Search for revisions by date in the format month/day/year.

- Operator — Search for revisions by operator name. Enter the name of the operator that created the revision. For example, SYSTEM ADMINISTRATOR.
- SCR# — Search for revisions by SCR number. For example, 42.

Purge revisions

Applies to User Roles: System Administrator

Purge scripts help administrators with revision maintenance. The `sc.revision.purge.hanging` script purges all revisions that no longer have a parent record because the parent record was deleted or renamed. The `sc.revision.purge` script purges all revisions from the system.

There are two options available when purging revisions.

To search for revisions:

- **Purge All Revision Records** removes all revisions from your system.
- **Purge Hanging Revisions** removes only hanging revisions from your system. A hanging revision is a revision with no associated file or form. For example, if you create a form, create a revision of the form and then delete the original form, the revision remains on the system but with no associated file or form.



Note

Note You can restore a deleted form from its revision file.

1. Sign in as a System Administrator.
2. Click **System Administration** > **Base System Configuration** > **Miscellaneous** > **Purge Hanging Revision Records**.
3. To remove hanging revisions from your system, click **Purge Hanging Revisions Records**.
4. To remove all revisions from your system, click **Purge All Revision Records**.

1.1.2. Best practices for promoting customizations

The following items are vital in any environment for successful customization of a Service Manager system:

- The process has to be defined and consistently applied.
- All changes have to be done in one system, the development system, and must be thoroughly documented.
- A single unload should contain all records that were customized. In some environments, many developers work on developing customizations. In such environments, a central system should be used to load the individual changes, using revision tracking. That central system can then be used to create the unload.
- All changes must be tested in a central system, the test system, and issues found there must be repaired in the development system.

Have a well-defined process

It is important that you have a well-defined and documented process for proceeding from development through test into production. A well-defined process will promote user satisfaction and prevent delays in the development cycle.

Document all changes

It is the best practice that you document each change made when you tailor the system, regardless of how the promotion will be done.

Part of the development process should include documenting all changes. Changes can be documented in an audit log that's stored outside of Service Manager, such as in an Excel® spreadsheet.

Another possibility is to use Service Manager Change Management for keeping track of all changes done during the development process.

Important: You must keep track of every change done to the development system, so that no change is lost when creating the patch unload.

Create a single unload

After the customizations are completed on the development system, create a single unload to apply to the test system.

Warning: If you create and apply multiple unloads, you increase the possibility that records can be overwritten or lost.

Use a development system

Make all changes in the development system. Making changes in the test system or directly to the production system will cause the systems to become out-of-sync and more difficult to maintain. Applying changes to a system that's out-of-sync with the system on which the changes were based will most likely result in broken functionality.

Use a test system

Use a test system to apply the previously-created unloads and test their functionality. If problems are found during these tests, they can be fixed in the development system where a new unload will be created. Repeat this cycle until the test is successful.

After testing is complete, you can apply the tested unload to the production system. Some companies require training before an unload is released to production. You can apply the tested unload to a training system first, and then to production. The benefit of having a single tested unload is that it can be applied to as many systems as necessary.

Related topics

Environment configuration
Tools to use for promoting customizations
Development auditing
Differential Upgrade utility
Unload script utility
Revision control
Promote customizations from development to production

1.1.3. Promote customizations from development to production

1. Before you promote customizations from development to production, thoroughly document every change made in the development system that will need to be part of the patch that's promoted to production. The information that you collect in this documentation depends on the environment, but should at a minimum include the following fields, which are mentioned in the log file:
    - Change Number
    - Request Description
    - Task Number
    - Task Description
    - Change Requester
    - Change Owner
    - Record modified
    - Date Modified
    - Change Description
    - Unload Name
    - Patch Name
    - Test plan

    You can document changes either inside of Service Manager Change Management or by using an external program such as Microsoft Excel.
2. Test all changes in the development system.
3. After all changes are documented, tested, and functioning in the development system, create a single unload. We recommend that you:
    1. Remove the field "keys" from the exclude list in the signaturemake file for the dbdict table before creating the signatures on both production and development. That way, dbdicts that had changes to the keys will be unloaded into the file that's loaded into test and then production and these changes are easier to find.

2. Use the Differential Upgrade utility to create a single unload file containing all changes.
3. Manually modify all keys that were changed in any of the dbdicts contained in the unload

4. After the single unload is created, perform the following steps:
    1. Apply it to a test system that's a recent copy of the production environment.
    2. Test the changes thoroughly.
    3. Document any reported issues and fix these in the development system after each test iteration.
    4. Provide a repaired patch file (while still using a single unload file) to the testers at the beginning of each test iteration.
5. After the unload is thoroughly tested and accepted, the latest unload can be applied to a production system or to any other system that needs to be upgraded with the patch, such as a training system.



Note

Note

Do not modify the patch after this point. Address new issues found after testing is complete in the next development cycle.

1.2. Data management

These topics focus on data management within the system. The main utilities that comprise data management are:

- Cascade updates — alters the data in one or more dependent files to match changes made to data in a source file.
- Data Policy — applies values, sets mandatory fields, and validates at the table level.
- Data validation — defines a field's valid values.
- Global lists — predefined lists within the system.
- Customizing the Help system — provides administrators with instructions on how to customize the online Help system.
- Sequential numbers — used in conjunction with Format Control to generate sequence numbers for records in a database.
- Stored queries — retrieves and displays information by using predefined search parameters.

Related topics

Tailoring
Cascade Updates
Data Policy
Data validation
Global lists
Sequential number setup
Stored queries

1.2.1. Fields and Keys tab

The Fields and Keys tab describes the properties of all fields and keys in a Service Manager table. For administrators, this tab gathers field-related tasks into a central point of access.

The Fields and Keys tab contains the following sections and links. Some sections or links may not appear unless they relate to the selected field.

Fields section

View, add, delete, or edit a field or array. A question mark decorator on the field icon indicates that there is a default help record for the field.

Button or property	Button functionality	Comments
New field	Create a field in the tree.	Each field must a have Service Manager data type. To create a new array use the New Array option first and then add fields to the array.
Delete	Remove a field.	Unavailable if the table contains records or the field is a key.
New array	Create an array and a child node (inner field) that has the same name and an unspecified data type.	You must choose a type for the child node after you save the array.
Search field	Search for a field or alias that contains a specified string value.	Look for the field by name, partial name, index value, alias, or Web Services API name.

Keys section

View, add, delete, or edit keys.

Button or property	Button functionality	Comments
New	Create a new empty key.	You must choose a key type after you save it.
Delete	Remove a key.	
Up	Move the key position up.	Not available for the first key.
Down	Move the key position down.	Not available for the last key.

General properties section (fields and arrays)

View or edit the Service Manager definition of fields and arrays. The System Definition utility displays these properties when you select a field.

Button or property	Button functionality	Comments
Name	Unique field name.	
Type	Specify the Service Manager data type of the field.	The Service Manager data types are abstractions of the RDBMS data types.
Index	Identifies the order field within the Service Manager definition.	The index of an unsaved field is -1 until you save the field.
Caption	Specify an optional default label for this field.	Forms Designer uses this value to create default column labels and chart titles. If you omit this information, Forms Designer creates a caption from the field name.
Unique in domain	Appears only for a field in a structured array.	
Rename the field	Click to change the field name. Proceed with caution. Renaming a field can invalidate existing queries.	If Service Manager has add permissions on the back-end RDBMS, it automatically creates the new field. Alternatively, the Service Manager administrator can provide the RDBMS administrators with DDL for field name change.
Create/Edit default help on this field	Click to open the field Help Editor where you can view, add, delete, or edit field Help content. Service Manager accesses this help record whenever the field appears in a form.	
Search for specific help on this field in forms.	Link to the field Help Editor where you can click **Search** to view a list of other help records that will override the default help record for the field. The best practice is to create only one help record for each field. If you customize the same definition for each form where the field appears, field help maintenance increases dramatically.	

General properties section (keys)

View or edit the Service Manager definition of keys. The System Definition utility displays these properties when you select a key.

Button or property	Button functionality	Comments
Type	Specify the data type of the key.	Be sure to understand the data in the field before you change the Service Manager field type so that you can select a compatible data type. Hewlett-Packard recommends that you don't change the data type for any unique keys in out–of–box tables.
Add	Add a field to the key from a list of valid fields.	If Service Manager has add permissions on the back-end RDBMS, it automatically creates a new index based on the key. Alternatively, the Service Manager administrator can provide the RDBMS administrators with DDL for the new index.
Remove	Remove a field from the key.	If Service Manager has add and drop permissions on the back-end RDBMS, it automatically removes the existing index corresponding to the key and creates a new index. Alternatively, the Service Manager administrator can provide the RDBMS administrators with DDL for the new index after the key has been removed.
Up	Move the field up one level within the key.	If Service Manager has add and drop permissions on the back-end RDBMS, it automatically removes the existing index corresponding to the key and creates a new index. Alternatively, the Service Manager administrator can provide the RDBMS administrators with DDL for the updated index.
Down	Move the field down one level within the key.	If Service Manager has add and drop permissions on the back-end RDBMS, it automatically removes the existing index corresponding to the key and creates a new index. Alternatively, the Service Manager administrator can provide the RDBMS administrators with DDL for the updated index.

Aliases section

View, add, delete, or rename an alias for a field.

Button or property	Button functionality	Comments
New	Creates a separate name (alias) that Service Manager applications can use to find the field.	Field aliases must be unique within a table.
Delete	Removes the selected field alias.	Deleting an alias doesn't delete the original field.
Rename	Renames the selected field alias	Field aliases must be unique within a table.

SQL Mapping section

View or edit the SQL mapping of Service Manager fields in a back-end RDBMS.

Button or property	Button functionality	Comments
SQL table alias	Type the 3 character name that Service Manager uses to identify what type of back-end RDBMS table the field definition relates to. For example, m1 refers to the first main table and a1 to the first alias table.	Service Manager uses this name to look up the actual table name from the RDBMS.
SQL field name	Type the name of the column in the back-end RDBMS you want to map to this field.	Make sure that the you don't use any special characters or reserved words that your RDBMS excludes.
SQL data type	Type the RDBMS data type you want to use for this field. You can leave this value blank for new fields and Service Manager will assign a default data type.	Do not include the length as part of the data type. Use the SQL data length field for this information.
SQL data length	Type the number characters you want this field to use. You can leave this value blank for new fields and Service Manager will assign a default data length.	Do not include the RDBMS data type as part of the field length. Use the SQL data type field for this information.

Data policy section

View or edit data policy rules for a field.

Button or property	Button functionality	Comments
Default value	Specify a default value for the field.	Not valid for a field in a structured array. Enter an expression by preceding it with a carat (^). For example, ^operator() specifies that the field contains the current user log-on name.
Data encrypted	Encrypt the data in the field.	Not valid for a field in a structured array.
Match table	Specify the name of a table that has a key relationship with the new field.	Not valid for a field in a structured array.
Match field	Specify the name of the field within the match table that has a key relationship.	Not valid for a field in a structured array.

Editing rules section

Describes the rules for viewing or editing a field. Specify true, false, or a Boolean expression.

Button or property	Button functionality	Comments
Mandatory	The field must appear if the value is true.	Not valid for a field in a structured array.
Validation rule	Specify the Boolean expression that must be true for the field to appear.	Not valid for a field in a structured array.
Read-only	The field can't change if the value is true.	Not valid for a field in a structured array.
Invisible	The field must not appear if the value is true.	Not valid for a field in a structured array.
Available in lists	The field must appear in a list if the value is true.	Not valid for a field in a structured array.

Structured array properties

Create a separate table that you can access independently. For example, the computer table contains structured arrays that you can access directly through **System Definition** > **Table Editor**.

Button or property	Button functionality	Comments
Separate table	Check to create a table that you can access through the Table Definition section of the System Navigator.	Valid only for a field in a structured array.
Unique table	The name of the separated table. This name is *parent_table_name+structured_array_name*+uniq. For example, computerdriversuniq.	Valid only for a field in a structured array.
Attribute table	The name of the attribute table. This name is *parent_table_name+structured_array_name*+attr. For example, computerdriversattr.	Valid only for a field in a structured array.

IR properties

View or edit information retrieval rules for the table. This section appears only when you select the IR key definition for the table.

Button or property	Button functionality	Comments
Condition	Specify a Boolean expression that must be true if a table is searchable.	
Detail form	Specify the form that displays any record in the list generated by a query.	
List form	Specify the form that displays the record list generated by a query.	
Return field	Specify the name of the field to be written back to the data source.	

If you make any changes to fields or keys, click **Save**.

1.2.1.1. Add a field to a table

Note: You must use the Windows client whenever you need to add a new field/key to a database dictionary table.

1. From the System Navigator, click **Tailoring** > **Database Dictionary**.
2. Place the cursor in the File Name field, and then click **Search**.
3. Select any table from the record list.
4. Click the **Fields** tab.
5. Place the cursor in a type structure to add a field to the structure.
6. Click **New Field/Key**.
7. Specify information for all required fields.
   • Choose a data **Type** from the list. For an array, choose the appropriate data type if all elements are the same type. Otherwise, choose **Text** for an array of different data types, or choose **Structure** for an array of structures.
   • Type a field **Caption**.
   • Click **Rename field** if you decide to change the name of the field.
   • Specify **Data policy** settings as needed for **Default value**, **Validation rule**, and **Data encrypted** (encryption status).

• Specify **Match table** and **Match field** values to define or enforce foreign key relationships. For example, you can specify that the contact.name field in one table must match a valid contact.name field from another table.
• Specify **Editing rules** (display properties) for **Mandatory** display, **Read-only**, **Availability**, and **Invisibility**.
• If necessary, specify one or more **Alias fields** with a different name but the same index, level, and type as a selected field.

8. Click **Save**.

Tip: You can edit any field or key directly from the System Navigator. Double-click the field or key to view the details with the Table Editor.

Related topics

System Definition
Table tab
Fields and Keys tab
Edit a field in a table
Edit a record key

1.2.1.2. Edit a field in a table

Note: You must use the Windows client whenever you need to add or edit a field/key in a database dictionary table.

1. From the System Navigator, click **Tailoring** > **Database Dictionary**.
2. Place the cursor in the File Name field and click **Search**.
3. From the System Definition utility overview, click the **Fields** tab.
4. Click **Edit Field/Key**.
5. If necessary, click the **plus sign** (+) to expand array structures.
6. Click a field to view its general properties, data policy, and editing rules on the right side of the window. Add or update any of the following settings.

General properties

Field	Description
Type	You can change the type if the field is one of these types: Boolean Date time Decimal Text Changing the data type can produce unpredictable results. Proceed with caution.
Caption	Field caption
Rename the field	You can rename the field; however, proceed with caution. Renaming a field can invalidate existing queries.
Create (or Edit) default help on this field	Create a new Help record or edit the existing Help record.
Search for specific help on this field in forms	You can search for an existing field help topic to apply to this field. Click **Search** to generate a record list of available field help topics.

Aliases

Field	Description
New	Click **New**, type a new alias name, and click **OK**.
Delete	Select an existing alias and click to remove it.
Rename	Select an existing alias, click **Rename**, type a new alias name, and click **OK**.

Data policy

Field	Description
Default value	A default value for the field is optional.
Data encrypted (encryption status).	Select or clear the checkbox.
Match table Match field	Define or enforce foreign key relationships. For example, you can specify that the contact.name field in one table must match a valid contact.name field from another table.

Editing rules

Field	Description
Mandatory	Type **true** if the field must appear on the form.
Validation rule	Specify a condition that's true or false.
Read-only	Type **true** if users can't change the field value.
Invisible	Type **true** if the field should be invisible.
Available in lists	Type **true** if the field should appear in a list.

7. If you make any updates, click **Save**.

Tip: You can edit any field or key directly from the System Navigator. Double-click the field or key to view the details with the System Definition utility.

Related topics

Fields and Keys tab
Add a field to a table
Edit a record key

1.2.1.3. Edit a record key

Applies to User roles: System Administrator

We recommend that you don't change the data type for any unique keys in out–of–box tables.

To edit a record key:

1. Click **Tailoring** > **Database Dictionary**.
2. Type a file name and then use search to find the table.
3. Select a record to edit.
4. Click the **Keys** tab to edit.

5. Click a key to view or change its general properties.
    - Change the key type as needed. Proceed with caution if you change the key type to a more restrictive value.
    - Click **Add** or **Remove** to change the content of the key.
    - For IR keys, you can also edit the associated IR properties.
6. To add a key, click **New**. A new key position appears in the Keys tree structure.
7. To remove a key, click **Delete**.
8. If a key is eligible to change position, click **Up** or **Down**.

1.2.1.4. Update the size of a field

Applies to User Roles:

System Administrator

To update the length of a field in a table:

1. Click **Tailoring > Database Dictionary**.
2. Type the name of the table in File Name. For example, use rootcause to update the size of a field in this table.
3. Click **Search**.
4. Select the field to update, dump for example.
5. Change the field SQL type from VARCHAR(30) to VARCHAR(100), for example.
6. Click **OK**.

1.2.2. Cascade Updates

The Cascade Update utility allows Administrators to maintain database integrity and consistency by altering or deleting the data in one or more dependent files to match changes made to data in a source file.

For example, incident records are linked to the contacts table by the contact name (probsummary, contact.name = contacts, contact.name). If someone were to edit the contact record and change contact.name, the associated incident records would lose their link and become orphaned. You could set up a Cascade Update record to monitor the contact.name field in contacts and change the contact.name field in related incident records to match accordingly.

Cascade Update example

In this Cascade Update example DEFAULT HQ has moved and now needs to update all of their contact records to reflect the new location "DEFAULT CORPORATE". Use the Cascade Update Utility to automatically update all DEFAULT HQ contact records when the location has been updated in the DEFAULT HQ location record.

There are two phases to make this happen:

- Add a trigger record.
- Add a Cascade Update Configuration record.

1. Set a trigger to monitor the location file and call the RAD application, cascade.update.wrapper, when the location record is updated (After Update) in the location file.
2. Configure a Cascade Update Configure Record to update the location in all contact records associated with that same location.
3. Test the Cascade Update.

Create a Cascade Delete

To set up a Cascade Delete between two related tables (for example, probsummary and problem):

1. Open the erddef table using Database Manager.
2. Select or create the erddef record that defines the relationship between the two tables.

3. Select the **Cascade Delete** option.
4. Save and Exit.

Alternatively, you can use the Cascade Update tool and follow the instructions in that tool to set up the Cascade Deletes there.

Create a Cascade Update

To create a Cascade Update:

1. Create a trigger record to monitor the source table.
2. Create a Cascade Update configuration record to set parameters for the update.

Create a Cascade Update/Delete Configuration Record

1. Click **Tailoring** > **Tailoring Tools** > **Cascade Updates**.
2. Click **Search**.
   A list of cascade update/delete configuration records opens.
3. Select a configuration record to open it.
4. Fill in all necessary fields.
   Note: For more detailed information on the fields, insert your cursor in that field and press **Ctrl+H**.
5. Click **Save** to save changes to an existing record.
   Caution: If you are attempting to add a new record from an existing record, make sure that you don't click Save because doing so will replace the existing record with the new record. If you are adding a new record, click Add.
6. Click **Add** to add a new record.

Create the example Cascade Configuration Record

1. Click **Tailoring** > **Tailoring Tools** > **Cascade Updates**.
   The Cascade Update/Delete Config Record opens.
2. Type the following information in the Cascade Update/Delete Config Record:
   • **Name**: location
   • **Type**: After Update (Configuration Rules)
   • **Target File**: contacts
   • **Execute Condition**: location in $L.new~=location in $L.old
   • **Update Type**: Cascade Update
   • **Source Key**: location
   • **Target Key**: location
   • **Source Fields**: location
   • **Target Fields**: location
3. Click **Add**.
   Service Manager displays the message: Cascade Update Config record updated.

Create the example trigger record

1. Click **Tailoring** > **Database Manager**.
2. In the Form field, type: `triggers`
3. Press **Enter**.
   The trigger record opens.
4. Type the following information in the triggers record:
   • **Trigger Name**: update.contact.location
   • **Table Name**: location
   • **Trigger Type**: 4 - After Update
   • **Application**: cascade.update.wrapper
   If you are configuring a Cascade Update for device records, type **am.cascade.update.wrapper** in the **Application** field. This wrapper works with the *device joindef* files.
5. Click **Add**.
   The message line indicates that the trigger record was added.

Test the example Cascade Update

1. Click **Tailoring** > **Database Manager**.
2. In the **Form** field, type the following: `contacts`
3. Press **Enter**.
   Service Manager displays a list of forms matching your search criteria.
4. Double-click contacts. The contacts.g form opens.
5. Click **Search**.
   Service Manager displays a list of all contact records.
6. From the pull-down options menu, select **Modify columns**.
7. Use the drop-down arrow to change the **company** field to **location**.
8. Click **Proceed**.
9. Click in the heading of the location column to sort the list so that all contacts with the same location are shown together.
10. Open a second session of Service Manager.
11. Click **Tailoring** > **Database Manager**.
12. In the **Form** field, type the following: `=location`.
13. Click **Search** or press **Enter**. The Search location records form opens.
14. Click **Search**.
    Service Manager displays a list of all location records.
15. Double-click on an ACME HQ record in the record list.
    The ACME HQ location details form opens.
16. Change the location by typing **ACME CORPORATE** in the **Location** field.
17. Click **Save**.
    Service Manager displays the message: location record updated.
18. Revert to the first session of Service Manager, which is still showing the record list of contact records.
19. Click **Refresh**.
20. All of the ACME HQ locations are now ACME CORPORATE.

1.2.3. Data Policy

Data Policy enables System Administrators to apply default values, mandatory fields, and lookup validations to a specific table. These policies, once set, are enforced across the entire system, regardless of what form is being used to display the data.

Format Control performs many common data tailoring tasks in Service Manager, is often complex, and is applied at the form level. If overused, Format Control can affect system performance. Data Policy operates at the table level and achieves many of the same results as Format Control without the complexity and without taxing system resources.

Related topics

Environment configuration
Data Policy expressions
Data Policy and the object record
Data Policy and encryption
Access Data Policy
Change the encryption key value
Example: Create and manage a revision
Create a Data Policy revision

1.2.3.1. Access Data Policy

Applies to User Roles: System Administrator

To access Data Policy from the System Navigator

1. Open **System Definition > Tables** > *file_name* > **Fields** > *field_name*.

The **Fields and keys definitions for the application table** form opens.

2. Edit the field properties in the Data Policy section, and then click **Save** to save your changes.

To access Data Policy from the Database Dictionary utility

1. Open the table (File Name) that you want to set Data Policy for in the database dictionary.
2. Select **Data Policy** from the More Actions menu.
   The datadict.g form opens, containing the selected file.
3. Edit the file properties, and then click **Save** to save your changes.

To search for Data Policy records

1. Open **Tailoring** > **Data Policy**.
2. Specify search criteria to select the records you want, or leave the form blank to see all records, and then click **Search**.
3. Select a table (file) from the list.
4. Select **Data Policy** from the More Actions menu.
   The datadict.g form opens, containing the selected file.
5. Edit the file properties, and then click **Save** to save your changes.

Related topics

Environment configuration
Data Policy
Data Policy expressions
Data Policy and the object record
Data Policy and encryption
Change the encryption key value
Example: Create and manage a revision
Create a Data Policy revision

1.2.3.2. Data Policy expressions

Data Policy rules apply to the GUI presentation of data. This allows some degree of data control into your system without the need to construct numerous different forms and views.

For example:

- In the Invisible or Read Only fields, you specify an expression. If this expression evaluates to true at the time the record is opened, any controls referencing the field in question are set to read-only or visible, as appropriate
- If you want only System Administrators to be able to modify the contact.name field in the contacts file, use this expression: `index("SysAdmin", $lo.ucapex)>0`

The record's database dictionary record defines the fields available on a file's Data Policy record. You can't add new fields directly to a Data Policy record.

To see your changes to Data Policy you must restart the client by logging out of the system and logging in again.

1.2.3.3. Usage Type column in Data Policy

The Usage Type column determines a field's availability for defining views, defining templates, mass functions, and smart actions in Service Manager.

Usage Type	Description
Deprecated	The field won't be available for defining views, defining templates, mass functions, and smart actions. In general, the Deprecated usage type also indicates that you may use the field in older versions but now this field is obsolete. It's still there for upgrades. However, since it's no longer used, you can use this usage type to exclude it from views.
System	The field will only be available for defining views.
Application Data <empty>	The field isn't restricted. There is no functional difference when you leave the value empty or set the value to **Application** or **Data** (informational only)

You need to leave the Usage Type column empty in Data Policy. Or, set the value to **Data** or **Application** for any custom fields used with Views, Templates, mass functions, or smart actions.

1.2.3.4. Data Policy and the object record

Data Policy control for a table can associate and expand with an object record. If there is an associated object, the object offers control over revisions, IR searches, and record displays through SM Manage. If a record in Data Policy has an object associated with it, the fields on the Engine Specifications tab are grayed out and unavailable. A button appears that takes you to the object record, where you can make changes.

If a Data Policy record doesn't have an associated object, then the Engine Specifications and SM Manage tabs contain fields you can edit that control revisions and display.

Related topics

Environment configuration
Data Policy
Data Policy expressions
Data Policy and encryption
Access Data Policy
Change the encryption key value
Example: Create and manage a revision
Create a Data Policy revision

1.2.3.5. Creating revisions

The revisions option for Data Policy creates a revision database dictionary record, whose name is specified on the Engine Specifications tab. You can create revisions either for an entire set of records or for a single form or record. You must be at the start panel of the application you are working with to create baseline revisions for an entire set of records. For example, you create a baseline revision of all the forms in the Forms Designer from the Forms Designer main menu.

When you create a revision, you create a copy of the record only, not the associated database dictionary record. As a result, any time a field is added to a database dictionary record, you add it to the revision file database dictionary record.

Create a Data Policy revision

Applies to User Roles: System Administrator

To create a Data Policy revision:

1. Click **Tailoring** > **Web Services** > **Data Policy**.
2. Click the **Engine Specifications** tab.

3. Type a revision file name in the **Revision File Name** field.
4. To limit the number of revisions allowed, fill in the **Max # of Revisions** field.
5. Click **Create Revision File** from the More Actions menu.

Example: Create and manage a revision

Applies to User Roles: System Administrator

The following example steps through the process of creating and controlling a revision. The example uses the location file.

1. Open the location record in Data Policy.
2. Type a revision name on the Engine Specifications tab, for example type `LocationRevision`.
3. Type the maximum number of revisions allowed on the system for files controlled by this record. For example, type `2`.
4. Save the location Data Policy record.
5. Go to the Database Manager and type `location` in the Form field. A record list opens.
   Every file in the record list is controlled by the Data Policy record Locations.
6. Select a record, for example ACME HQ, based in Chicago.
7. With the record open, click **Revision > Create Revision** from the More Actions menu. Include any relevant text to help you understand the revision and its purpose. Label this revision one.
8. Make a change to the ACME HQ record and save.
9. Click **Revision > Create Revision** from the More Actions menu. Label this revision two.

The maximum number of revisions has now been reached for the files controlled by the Locations Data Policy record. The next revision you make will eliminate the revision you labeled one.

Related topics

Environment configuration
Tools to use for promoting customizations
Revision control
Data Policy and encryption

1.2.3.5.1. Example: Create and manage a revision

Applies to User Roles:

System Administrator

The following example steps through the process of creating and controlling a revision. The example uses the location file.

1. Open the location record in Data Policy.
2. Type a revision name on the Engine Specifications tab, for example type `LocationRevision`.
3. Type the maximum number of revisions allowed on the system for files controlled by this record. For example, type `2`.
4. Save the location Data Policy record.
5. Go to the Database Manager and type `location` in the Form field. A record list opens.
   Every file in the record list is controlled by the Data Policy record Locations.
6. Select a record, for example ACME HQ, based in Chicago.
7. With the record open, click **Revision > Create Revision** from the More Actions menu. Include any relevant text to help you understand the revision and its purpose. Label this revision one.
8. Make a change to the ACME HQ record and save.
9. Click **Revision > Create Revision** from the More Actions menu. Label this revision two.

The maximum number of revisions has now been reached for the files controlled by the Locations Data Policy record. The next revision you make will eliminate the revision you labeled one.

1.2.3.5.2. Create a Data Policy revision

Applies to User Roles:

System Administrator

To create a Data Policy revision:

1. Click **Tailoring** > **Web Services** > **Data Policy**.
2. Click the **Engine Specifications** tab.
3. Type a revision file name in the **Revision File Name** field.
4. To limit the number of revisions allowed, fill in the **Max # of Revisions** field.
5. Click **Create Revision File** from the More Actions menu.

1.2.3.6. Revert to a previous revision

Applies to User Roles:

System Administrator

To revert to a previous revision:

1. Open the form you want to use a previous version of.
2. Click **More** or the More Actions icon.
3. Click **Revision**.
4. Click **Revisions > Find Revisions**.
5. Click the revision you want to restore.
6. Click **Revert to this Revision** from the More Actions menu. The system prompts you as to whether you want to save the current version of the record as a revision. It is recommended you do.

1.2.3.7. Purge revisions

Applies to User Roles: System Administrator

Purge scripts help administrators with revision maintenance. The `sc.revision.purge.hanging` script purges all revisions that no longer have a parent record because the parent record was either deleted or renamed. The `sc.revision.purge` script purges all revisions from the system.

There are two options available when purging revisions.

To search for revisions:

- **Purge All Revision Records** removes all revisions from your system.
- **Purge Hanging Revisions** removes only hanging revisions from your system. A hanging revision is a revision with no associated file or form. For example, if you create a form, create a revision of the form, and then delete the original form, the revision remains on the system but with no associated file or form.



Note

Note You can restore a deleted form from its revision file.

1. Sign in as a System Administrator.
2. Click **System Administration** > **Base System Configuration** > **Miscellaneous** > **Purge Hanging Revision Records**.
3. To remove hanging revisions from your system, click **Purge Hanging Revisions Records**.
4. To remove all revisions from your system, click **Purge All Revision Records**.

1.2.3.8. Data Policy and encryption

When the Data Policy for a table changes, the system checks to see if the encryption status changes. If the status changes from false to true, each record in the file is read, the field is encrypted, and the data is written back to the file. If the status has changed from true to false, each record in the file is read, the field is decrypted, and the data is written back to the file. The entire process includes reading and updating each record in the file, which could result in performance loss.



Important

Important If you encrypted any field in your database, ensure that you store the KEY value defined in the sm.ini file in a safe place. Without the correct KEY value, you will not be able to decrypt. Once a field in a table is encrypted, users can no longer search for the contents of the field. The field is not searchable.

Related topics

Environment configuration
Data Policy
Data Policy expressions
Data Policy and the object record

1.2.3.9. Change the encryption key value

You can set the KEY value that encrypts data on your system with the `encryptionkey` parameter in the sm.ini file. This field must be exactly 8 bytes in length. If you change the KEY value, then every field in every file must be checked and re-encrypted. Changing the KEY value results in a significant performance hit to the system while re-encryption takes place.

To change the encryption key value, perform the following steps:

1. Shut down the Service Manager server.
2. Run the `sm -changeencrkey:XXXX` command to change `encryptionkey`. For example, `sm -changeencrkey:qwertyui` where `qwertyui` is the new 8-byte key.
   This command decrypts all encrypted fields using the key defined in the sm.ini file and then re-encrypts those fields using the key specified in the command line parameter `encryptionkey`. The length of time the conversion takes depends on the size of the database and the number of encrypted tables.
3. Update your sm.ini file to the new key immediately after performing *step 2*.
   Note that if you are working in a horizontal scaling implementation, you must update the `encryptionkey` value in the **sm.ini** file for each host. Make sure the `encryptionkey` value is correct and consistent on each host. Correct `encryptionkey` for all nodes as required, and then restart the Service Manager server after updating the **sm.ini** file.

Caution: Encrypting SQL data that's already mapped will increase the size of the data. Therefore, the existing SQL mapping and column definition may not provide enough space to store the whole encrypted value. Be sure to change the SQL data type to accommodate the new column size. If the encrypted value gets truncated, the value can no longer be decrypted.

Use this formula to calculate the new column length:

encrypted_length = (unencrypted_length + 12) * 2.

1.2.3.10. Create an alias for your custom CI display name field

Applies to User Roles:

System Administrator

In the out-of-box system, the **device** table uses the **display.name** field to store CI display names. However, your production system may use a custom field to store CI names. For this reason, in the **device** dbdict record, the **sm.device.display.name** field is created as an alias for the **display.name** field. If your **device** table uses a custom field to store CI names, you only need to specify it in the data policy record so that Service Manager automatically recreates an alias named **sm.device.display.name** for it.

Caution

Caution Before your proceed, make sure that your custom CI name field has a character data type, and is configured to be a No Nulls key and not a Unique key. If your custom field doesn't meet these conditions, it is unavailable for selection from the Display Field drop-down list in the **device** data policy record.

To create an alias for your custom CI display name field, follow these steps:

1. Open the **device** table from Database Dictionary.
2. From the **More** or More Actions menu, select **Data Policy** to open the data policy record.
3. In the **Display Field** field, select your custom CI display name field from the drop-down list, for example, **mycompany.ci.name**.

   Note

   Note When you change the Display Field value, Service Manager removes the alias from the previous field and adds it to the new field.

4. Click **Save** to save the data policy record.
5. Click **OK** twice to save the DBDICT record.
6. Open the **device** DBDICT record again, and verify that an alias named **sm.device.display.name** has been created for your custom field (that is, both the custom field and the alias have the same index number).
7. Execute the **refreshSubscriptionDisplayName** ScriptLibrary to update the CI information in subscriptions.

   Note

   Note CI information in subscriptions is derived from the Display Field defined in the **device** data policy record. This step is needed whenever you have changed the Display Field setting.

1.2.3.11. Define auto complete table columns

Applies to User Roles: System Administrator

For all fields that have auto complete enabled, their data policy record provides an **Auto Complete Table Columns** tab, which enables you to define the fields (columns) to display in the auto complete drop-down table for the field. By providing additional key information about the field, this feature helps users easily select the right record from a list of auto complete results.

Tip

Tip For a list of fields with auto complete enabled in the out-of-box system, see the **Out-of-box Comfill fields with auto complete enabled** section in Auto complete.

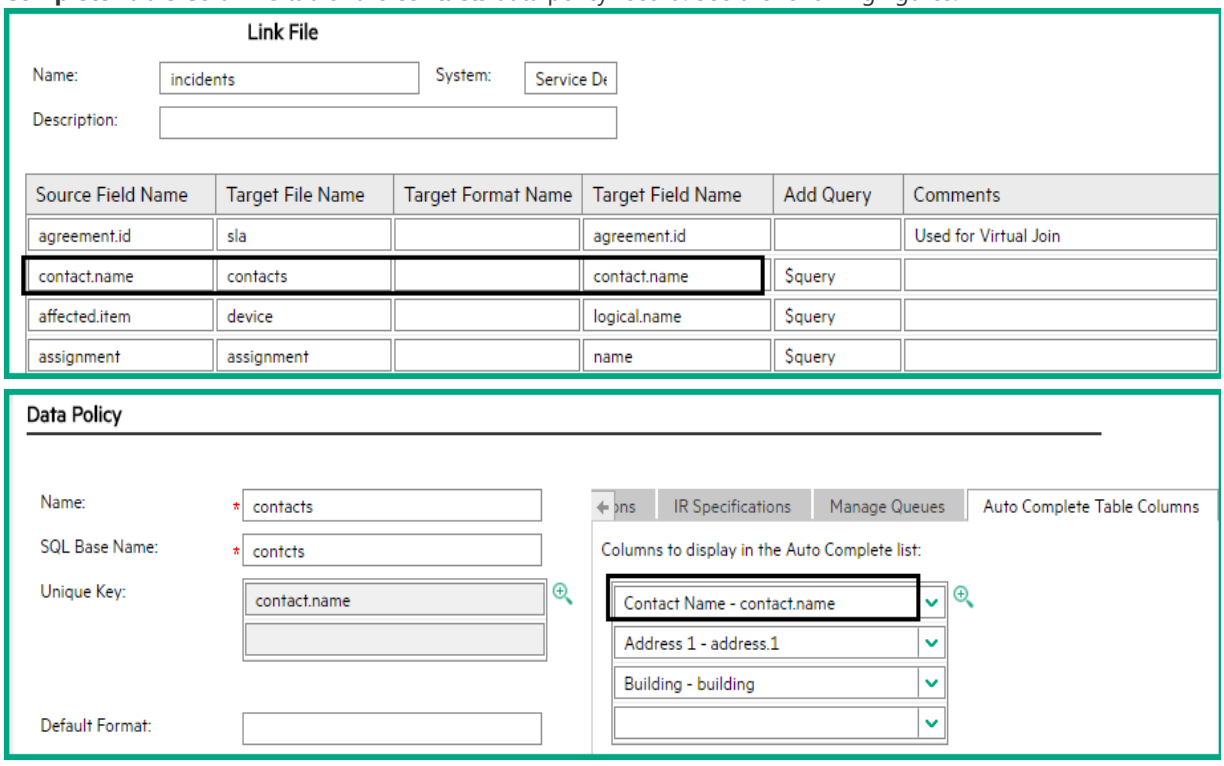
Note

Note Auto Complete is always enabled for a field that has a referenced table defined, regardless of the value of the comfillAutoComplete parameter and the Auto Complete property setting defined in Forms Designer for that field. For more information, see Auto complete.

For example, when a web client user enters CI information in the reference modules (such as Incident, and Change), a drop-down table for auto complete is displayed, which lists a number of field values of the matching CIs so that the user can easily select the right CI. By default, the Display Name, CI Identifier, Type, Environment, and Assignment columns are displayed in the auto complete table for CI fields, and only one column is displayed for non-CI fields (for example, the Contact field in an Interaction record).

You can customize the auto complete table columns for a field that has auto complete enabled. To do this, follow these steps:

1. Open the data policy record for the field. For example, if this is a CI field, open the **device** data policy.
   There are several ways to do this. For example, open the **device** dbdict record, and then click **More** > **Data Policy**.
2. Click the **Auto Complete Table Columns** tab, and then edit the existing list of fields.
   For a CI field, the auto complete drop-down table always displays the **display.name** field as the first column, even if you delete it from this tab.
   For a non-CI field, auto-complete by default uses the target field that's defined in the corresponding link record. If you want to add more columns, be sure to add this field as the first column. For example, for the **Contact** field in the Interaction form, the **Incidents** link record defines the **contact.name** field of the **contacts** table as the target field. Therefore, if you want to add more columns, you must add the **contact.name** field as the first column on the **Auto Complete Table Columns** tab of the **contacts** data policy record. See the following figures.



3. Save the data policy record.

> **Note**
> Note If you configure an empty or duplicate row, the system automatically removes it when you save your configuration.

1.2.3.12. Specify a referenced table for a field

The Data Policy for a table can define a referenced table for a field in the table. A referenced table is the table that's referenced by the current field. From a business logic perspective, the current field must be used to store the same data as the unique key field of the referenced table. For example, in the out-of-box **cm3r** Data Policy record, the **device** table is defined as the referenced table of the **assets** field (the Affected Configuration Item field). Once you have specified a referenced table, Service Manager creates a link that points from the current field to the unique key field of the referenced table.

Furthermore, the referenced table must have a Display Field defined. The current field displays the values of the display field, instead of values of the unique key field of the referenced table. For example, the Affected Configuration Item field in a Change record displays the CI display names (stored in the display.name field), instead of the CI Identifier values (stored in the logical.name field).

This functionality automatically updates the CI data in the reference modules (such as Change, and Incident) when a CI name is updated in the Configuration module so that data integrity is maintained across the modules.

To specify a referenced table for a field, follow these steps:

1. Click **Tailoring** > **Data Policy**.
2. Open the corresponding Data Policy record. For example, for the **affected.item** field in the **cm3r** table, open the **cm3r** Data Policy record.
3. Select the **Field Settings** tab.
4. Select the field, and select a value from the drop-down list in the Referenced Table column. For example, select **device**.

   Note

   Note Currently, only the **device** table can be used as a referenced table.

5. Click **Save** to save the Data Policy record.
   In this example, Service Manager creates a link between the field and the **logical.name** field of the **device** table, if the link doesn't already exist. In the link record, the **logical.name** field is the target field.

Next, you need to specify a display field for the referenced table. This is the field of the referenced table whose values will be displayed in the current field. The current field derives data from the display field based on the unique key field of the referenced table. For details, see Specify a display field for a referenced table.

1.2.3.13. Specify a display field for a referenced table

You need to define a Display Field in the Data Policy (datadict) record of a referenced table. For example, the out-of-box system defines the **device** table as the referenced table for the **assets** field of the **cm3r** table, and defines the **display.name** field of the **device** table as the Display Field. Because of this configuration, Affected CIs in a Change record are retrieved according to their CI Identifier values and displayed as their display names.

Note

Note A referenced table must have only one field as the unique key. Currently only the **device** table can be set as a referenced table.

To specify a display field for a referenced table, follow these steps:

1. Click **Tailoring** > **Data Policy**.
2. Open the Data Policy record for the referenced table. For example, open the **device** Data Policy record.
3. In the **Display Field** field, select a value from the drop-down list. For example, select **Configuration Item Name - display.name**.
   A display field has an alias defined in the dbdict record. When you change the Display Field value, Service Manager removes the alias from the previous display field and adds it to the new field.

   Note

   Note A display field must meet the following conditions:

   - Has a character data type
   - Is not a Unique key field.
   - Is a No Nulls key field

4. Click **Save** to save the Data Policy record.

1.2.4. Data validation

The validity table enables you to define a field's valid values in a list of values, a list of ranges, a secondary file, and customized RAD subroutines. These values are then used for validating operator-entered data and for lookup processing (similar to Service Manager Find and Fill). These various validity definitions are consolidated in one file, resulting in easy maintenance. This can be of significant benefit when multiple forms access one database.

Validation routine

The value of a field is validated against a list of values (for example 10, 20, 30, or 40), a list of ranges (for example 50 through 60), a secondary file (similar to the Format Control secondary file query) and customized RAD subroutines. The fields within a file are validated in a particular order (for example fieldx, fielda, fieldg), in alphabetical sequence (for example fielda, fieldg, fieldx) or according to a field validation sequence number. The default processing of the validation routine validates the value of a field against the definitions in the validity table. If the current value of a field is invalid, a summary list of valid values and ranges is presented to the user. If no values or ranges are defined for the field, a list of records from the secondary file is presented. The user can then correct the invalid value and continue processing; return to the calling application; or force the application to accept the invalid value. The validity table also enables you to override this processing and execute customized RAD code.

User-defined processing statements

After determining that a field contains a valid value or range, user-defined processing statements are executed. Validity processing continues to the next field in the validity definition table until all specified fields are validated.

Each validity table definition contains four levels of definition:

- Field Summary
- Secondary File Query Information
- Values Summary
- Range Summary

Maintenance procedures

When it is necessary to differentiate the maintenance procedures of these different levels, the order of the documentation is Field level, Secondary Queries level, Values level, and Ranges level. For instance, the section on updating shows how to update Validation Records at the Field Level, the Value Level, and then the Range Level. All applications and procedures described refer to a unique maintenance application that specifically supports validity table maintenance. If you access any of the validity table databases with any other application (such as Database Manager), you must ensure that all data is properly structured in order for the validity definitions to work correctly at run time. We recommend that you always follow the documented Validity procedures.

Related topics

Accessing the validity file
Deleting validity record components
Invalid application names
Invalid files and form names
Invalid field names
Printing validity definitions
Invoking validity table processing
Look-up processing
Validity validation rules

1.2.4.1. Accessing the validity file

Following are the ways you can access the validity file.

- Access Validity from a menu

1. Click **Tailoring** > **Tailoring Tools** > **Data Validation**. The Validity Table Specifications form opens.
2. Click **Search**. A list of validity maintenance records opens.

- Access Validity through Database Manager

1. Click **Tailoring** > **Database Manager**.
2. Type **validity** in the Form field, and then click **Search**. A list of the validity form records in your database opens.
3. Double-click a validity Format Name record, and then click **Search**. The Validity Table Specifications form opens.
4. Click **Search**. A list of data validation records opens.

- Access Validity from the command line

1. Click **Command** in the System Administrator's home menu.
2. Type `validity` in the command line.
3. Press **Enter**. The Validity Table Specifications form opens.
4. Type a field name or a file name in the form, and then click **Search**. The requested form opens.

Related topics

Data validation
Deleting validity record components
Invalid application names
Invalid files and form names
Invalid field names
Printing validity definitions
Invoking validity table processing
Look-up processing

1.2.4.2. Deleting validity record components

You can delete entire validity records or parts of records using several different methods.

- Delete the entire validity record and all its components.
- Delete a value or range definition from an individual record.
- Delete value and range definitions from a table format.

Related topics

Data validation
Accessing the validity file
Invalid application names
Invalid files and form names
Invalid field names
Printing validity definitions
Invoking validity table processing
Look-up processing
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table

1.2.4.3. Invalid application names

If you have entered an invalid application name in the **Alt Application** field in the **Alternate Application** tab, a record list of valid applications opens and the following message appears in the status bar: *The Application Name is invalid.* Select one from the list. Double-click on the correct application to replace the invalid entry.

Related topics

Data validation
Accessing the validity file
Deleting validity record components
Invalid files and form names
Invalid field names
Printing validity definitions
Invoking validity table processing
Look-up processing

1.2.4.4. Invalid files and form names

If any element of the **Files/Formats** field is not a valid database dictionary name or a valid form name, the following message appears in the status bar: `The name <name > is not a valid dbdict or format`. Search for database dictionary records. A record list of valid file names is displayed. Double-click on the correct file to replace the invalid entry.

If you have entered an invalid form name in the **Files/Formats** field and want to view a record list of valid forms, open the More Actions menu and select **Display All Formats**. A record list of valid forms opens. Double-click on the correct form to replace the invalid entry. If the form you select is not associated with a file, you must respond by indicating the file with which the form will be associated at run time.

Note: The file associated with the form must contain the **Field Name**. If it doesn't and the first element in the list is being validated, you are prompted to select a valid field name. If the second or greater element is being validated, you are returned to the original record with an error message.

To switch back to the list of files, select **Display Table List** from the More Actions menu.

Related topics

Data validation
Accessing the validity file
Deleting validity record components
Invalid application names
Invalid field names
Printing validity definitions
Invoking validity table processing
Look-up processing

1.2.4.5. Invalid field names

If you enter an invalid field name, the following message appears in the status bar: *The field name <field name > is invalid*. A list of valid field names for the file you have entered opens. Select a valid field from the list. The invalid entry is replaced by the field you have selected.

Related topics

Data validation
Accessing the validity file
Deleting validity record components
Invalid application names
Invalid files and form names
Printing validity definitions
Invoking validity table processing
Look-up processing

1.2.4.6. Printing validity definitions

A detailed report of a validity record has four main sections that display all the information contained in the record and in the detailed summary forms:

- Field specifications
- Query specifications
- Values specifications
- Range specifications.

Note: Any scalar field or array element that's greater than 60 characters in length is printed in its entirety with the use of continuation lines. All elements of all arrays are also printed. All continuation lines start with: ***

Related topics

Data validation
Accessing the validity file
Deleting validity record components
Invalid application names

Invalid files and form names
Invalid field names
Invoking validity table processing
Look-up processing

1.2.4.7. Invoking validity table processing

You can invoke validity table processing by calling the RAD application **validate.fields** in the **Subroutines** process of **Format Control**. Invoke validity table processing to achieve the following two goals.

- Validate the fields in a record during record processing.
- Allow look-up processing during record editing (similar to Service Manager Find and Fill capability).

Related topics

Data validation
Accessing the validity file
Deleting validity record components
Invalid application names
Invalid files and form names
Invalid field names
Printing validity definitions
Look-up processing
Validate fields during record processing

1.2.4.8. Look-up processing

Calling the validity routine while a record is being edited simulates the Find and Fill functions. This gives the user the ability to determine the valid values or ranges of a field before attempting to process a record.

You can execute the validity look-up routine from these three places.

- Format Control Additional Options
- Format Control subroutines
- Displayoptions

Related topics

Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.1. Format Control processes

There are eight major functional processes in Format Control that define the actions to be taken on a record. Each of these processes has a separate form within the Format Control Utility.

Main information — This form is the entry point in Format Control and has several functions.

- Initializes fields or variables that are later used in the Format Control record. Initialization expressions are the first operation performed for each evaluation of a Format Control record.
- Initially displays a value in a field when the record itself opens.
- Names special record list and initial query forms to use, sets up default sort sequences, and runs scripts.

Forms — This section in Format Control has two main functions:

- Specify alternate forms to display a list of records.
  Note: This option can't be used when the Record List (split screen) functionality is enabled.
- Specify alternate forms to display a single record.

The formatcontrol option allows you to display the information using different Service Manager forms based on conditions evaluated at run time. You can specify either QBE Forms to display a list of records or Alternate Forms to display a record. You may want to specify different lists according to the user role or capabilities.

By specifying alternate forms, whenever the condition is met, the user will have an option available called "Alternate Forms" where the user can select the form to use to display the information. Additional forms are useful to display different views such as financial or security information when the condition specified is met.

Queries — This process enables you to extract information from a file other than the primary file in order to perform calculations and validations and to report on information from more than one file.

Calculations — This process enables you to perform calculations on currently available fields or variables. The fields needed for calculations may be variables, fields in the primary file, or fields in any other secondary files that may have been queried.

JavaScript — This process enables you to call JavaScripts from Format Control.

Validations — This process enables you to set up a logical expression for checking data in fields or variables on the form. The validation expression you set up must evaluate to true upon the desired edit function for it to be successful. If the expression doesn't evaluate to true, a validation message opens, and the specified operation fails.

Subroutines — This process enables you to call RAD routines from Format Control.

Additional options — This process enables you to define menu options you want available to users on any form associated with the particular Format Control record. You can use Format Control to set up a menu option called Validity Lookup. This option performs a validity check on the field of focus in the form for which validity specifications have been defined. Invalid fields are highlighted and a message is displayed in the status bar giving details on the type of error incurred.
Note: This feature calls RAD subroutines and is available in Database Manager only.

Privileges — This process enables you to use security to control database options available to the user. If the contents of a field evaluate to true at processing time, the corresponding button is available to the user.

Related topics

Format Control

1.2.4.8.2. Displayoptions

You can create a Validity lookup option in the More Actions menu by creating a displayoption record. For detailed information on creating displayoptions, refer to the Display application help.

Use the following values in the **displayoption** record:

Field	Value
Default Label	Validity Lookup
Condition	true
RAD Application	validate.fields
Separate Thread?	true

Use the following values in the **displayoption** record:

Name	Value
name	cursor.field.name()
second.file	$L.filed
cond.input	val("true", 4)

Related topics

Data validation
Look-up processing
Format Control processes
Display application
Selecting display options
More Actions menu

1.2.4.8.3. Display value summary details

Applies to User Roles: System Administrator

You can display the details of a single entry in the values summary array, or view the details for all the entries displayed in a scrollable form.

To display the details of a single entry in the value summary array:

1. Place your cursor on a line in the value summary array.
2. Select **Options > Edit Line**. The details for the entry are displayed.

To display the details of all entries in the value summary array:

1. Place your cursor on a line in the value summary array.
2. Select **Options > Display All Values** in a validity record. Details of all the entries in the value summary array are displayed in a scrollable list.

Note: You may edit or sort the contents of this record as you would any table in Service Manager.

To display the typical table editing controls in the tool tray:

Select **Options > Edit Table**.

Related topics

Data validation
Format Control processes
Displayoptions
Display range summary details
Create validity table definitions
Add field-level definitions
Create validity table definitions

Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.4. Display range summary details

Applies to User Roles: System Administrator

You can display the details of a single entry in the range summary array, or view the details for all the entries displayed in a scrollable form.

To display a single entry in the range summary array:

1. Place your cursor on a line in the range summary array.
2. Select **Options > Edit Line**. The details for the entry are displayed.

To display the details of all entries in the range summary array:

1. Place your cursor on a line in the range summary array.
2. Select **Options > Display All Values** in a validity record. Details of all the entries in the Range Summary array are displayed in a scrollable list.

Note: You may edit or sort the contents of this record as you would any table in Service Manager.

To display the typical table editing controls in the tool tray:

Select **Options > Edit Table**.

Related topics

Data validation
Format Control processes
Displayoptions
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.5. Create validity table definitions

To create a new validity table definition, complete the appropriate records in the following order:

1. Add field-level definitions.
2. Add value definitions.
3. Add range definitions.

4. Validate the validity definitions.

Related topics

Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.6. Add field level definitions

1. Click **Tailoring** > **Database Manager**.
2. Type **validity** in the Form field, and then click **Search**.
   A list of the validity form records in your database opens.
3. Double-click **validity.summary**, and then click **Search**.
   The Validity Table Specifications summary form opens.
4. In the **Field Name** field, type the name of the field in which you want to create a validation record.
5. In the **Files/Formats** field, type the name of the file or files in which this field is found.
6. Click **Search** to search for records that may already exist with this field.
7. If no record exists, click **New** to add your new record.
   The following message appears in the status bar: Data Validation record added.
8. Complete the remaining fields in the form (including the other tabs) with the exception of the Values Summary and Range Summary arrays.
9. Click **More** or the More Actions icon, and select **Validate** to validate the information you have entered in the form.
   If you have entered an invalid value in the form, a message appears in the status bar specifying the error.
10. Click **Save** when you have successfully validated your record.
    A message appears in the status bar, stating that the Data Validation record has been updated.

Related topics

Accessing the validity file
Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record

Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.7. Add value definitions

Follow these steps:

1. Click **Tailoring** > **Database Manager**.
2. Type **validity** in the Form field, and then click **Search**. A list of the validity form records in your database opens.
3. Double-click **validity.summary**, and then click **Search**. The Validity Table Specifications summary form opens.
4. Put the cursor in the first line of the Values Summary array.
5. Click the Options menu, and choose **Edit Line**. The Validity Value Detail Specification form opens.
6. Complete the record as necessary, and then click **Add**.
   Note: It is possible that the same range could be allowed under multiple conditions within the same validity record; therefore, the system doesn't check for duplicate field values.
   Important: If the character field starts with a < or > symbol, you must insert a blank space before it. These symbols are dropped by the Service Manager parser, unless preceded by a blank space.
   The record is added and the buttons in the tool tray change. The following message appears in the status bar: Range/Value successfully added.
7. Click **Back** to return to the validity record.
   The system combines the values you entered in the Min Desc and Max Desc fields with the relational operator you specified to produce the string that appears in the Values Summary array.
8. Put the cursor in the next blank line of the Values Summary array and repeat the above steps until you have established all the ranges for your field.
9. Click **Save** to add the record to the validity file.

Related topics

Accessing the validity file
Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field level definitions
Create validity table definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.8. Add range definitions

Follow these steps:

1. Click **Tailoring** > **Database Manager**.
2. Type **validity** in the Form field, and then click **Search**. A list of the validity form records in your database opens.
3. Double-click **validity.summary**, and then click **Search**.
   The Validity Table Specifications summary form opens.
4. Put the cursor in the first line of the Range Summary array.

5. Click **More** or the More Actions icon, and then choose **Edit Line**. The Validity Range Detail Specification form opens.
6. Complete the record as necessary, and then click **Add**.
   For additional information on writing conditions and expressions, see information on system language in the related topics.
   Note: It is possible that the same range could be allowed under multiple conditions within the same validity record; therefore, the system doesn't check for duplicate field values.
   Important: If the character field starts with a < or > symbol, you must insert a blank space before it. These symbols are dropped by the Service Manager parser, unless preceded by a blank space.
   The record is added and the buttons in the tool tray change.
7. Click **Back** to return to the validity record.
   The system combines the values you entered in the Min Desc and Max Desc fields with the relational operator you specified to produce the string that appears in the Range Summary array.
8. Put the cursor in the next blank line of the Range Summary array and repeat the above steps until you have established all the ranges for your field.
9. Click **Save** to add the record to the validity file.

Related topics

Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.9. Validate the validity definitions

The validation routine ensures that field, file, form, and application names referenced in the definition are valid. It also prompts for a data type if you have left the **Field Type** field blank.

To validate the validity definitions:

1. Open a validity record.
   1. Click **Tailoring** > **Database Manager**.
   2. Type **validity** in the Form field, and then click **Search**.
      A list of the validity form records in your database opens.
   3. Double-click a validity Format Name record, and then click **Search**.
      The Validity Table Specifications form opens.
   4. Click **Search**.
      A list of data validation records opens.
   5. Select a record.
2. Click **More** or the More Actions icon, and choose **Validate**.

If the validation routine detects an invalid entry, a message appears in the status bar and a form opens with a list of field names to select the correct entry.

If all the record entries are valid, a message appears in the status bar stating that the validity record entries are valid.

Related topics

Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.10. Delete an entire validity record

Follow these steps:

1. Open a validity record.
    1. Click **Tailoring** > **Database Manager**.
    2. Type **validity** in the Form field, and then click **Search**.
       A list of the validity form records in your database opens.
    3. Double-click a validity Format Name record, and then click **Search**.
       The Validity Table Specifications form opens.
    4. Click **Search**.
       A list of data validation records opens.
    5. Select a record.
2. Click **Delete**.
   The following prompt is displayed: Are you sure you want to delete this record?
3. Click **Yes**.

Related topics

Accessing the validity file
Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.11. Delete a single value or range definition

Follow these steps:

1. Open a validity record.
    1. Click **Tailoring** > **Database Manager**.
    2. Type **validity** in the Form field, and then click **Search**.
       A list of the validity form records in your database opens.
    3. Double-click a validity Format Name record, and then click **Search**.
       The Validity Table Specifications form opens.
    4. Click **Search**.
       A list of data validation records opens.
    5. Select a record.
2. Place the cursor in the line containing the value or range definition you want to delete.
3. Click **More** or the More Actions icon, and choose **Edit Line**.
4. Click **Delete** in the detail form.
   The following prompt is displayed: Are you sure you want to delete this validity detail?
5. Click **Yes**. You are returned to the validity record.
6. Click **Save**.

Related topics

Accessing the validity file
Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field level definitions
Create validity table definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.12. Delete value or range definitions from a table

Follow these steps:

1. Open a validity record.
    1. Click **Tailoring** > **Database Manager**.
    2. Type **validity** in the Form field, and then click **Search**.
       A list of the validity form records in your database opens.
    3. Double-click a validity Format Name record, and then click **Search**.
       The Validity Table Specifications form opens.
    4. Click **Search**.
       A list of data validation records opens.
    5. Select a record.
2. Click **More** or the More Actions icon and choose **Display All Values** or **Display All Ranges**.
   A Validity Value Specifications form or Validity Range Specifications form opens.

3. Access the table editing controls. Click **More** or the More Actions icon, and choose **Edit Table**.
4. Put the cursor anywhere in the record you want to delete, and then click **Delete**.
   Warning: Make sure this is the record you want to delete. When you click Delete, you're not prompted to confirm the command. Instead, the record is immediately removed from the table.
5. Repeat the process with any other value or range definitions you want to delete.
6. Click **End** to exit the table edit mode.
7. Click **Back** to return to the validity record.
8. Click **Save**.

Related topics

Accessing the validity file
Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field level definitions
Create validity table definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.13. Print a detailed report of an entire validity record

Follow these steps:

1. Click **Tailoring** > **Database Manager**.
2. Type **validity** in the Form field, and then click **Search**.
   A list of the validity form records in your database opens.
3. Double-click a validity Format Name record, and then click **Search**.
   The Validity Table Specifications form opens.
4. Click **Search**.
   A list of data validation records opens.
5. Select a record to open it.
   Note: To generate a detailed report, the validity record must contain values or range definitions.
6. Place the cursor in a *summary* field containing a value.
7. Click **More** or the More Actions icon, and then choose **Edit Line**.
   A detail specification record opens.
8. Click **File** >**Print** or the Print icon.

Related topics

Accessing the validity file
Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions

Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.14. Validate fields during record processing

During record processing, the fields in a record can be validated one at a time or in a particular order.

Note: To invoke validity table processing, see the related topics. You must validate each validity table definition before using it to validate data records.

1. Access the Format Control record for the form which you want to invoke validity table processing. If a record doesn't already exist, you must create one. For details on accessing and using Format Control, see the related topics.
2. Display the Subroutines process.
3. Enter **validate.fields** in the **Application Name** field.
   Note: The logical values in the *Add*, *Update*, and *Before* fields are examples only. For information on how to decide when Format Control is processed, see the related topics.
4. Enter the applicable parameters for **validate.fields** in the **Name** and **Value** fields using the information from the following table:

Name	Value	Description
second.file	$file	Executes all validity table records (all fields) defined for the file being processed.
name	<field name>	Executes only one validity table definition (one field) defined for the file being processed. **Note**: You can repeat this type of definition in the Format Control subroutines section as many times as necessary. However, it is more efficient to call the subroutine once and pass a list of the fields to be validated.
names	<list of fields> or $<variable>	Executes a specific list of validity table definitions defined for the file associated with the form. For example, define a $ variable in either the Initializations or Calculations process of the Format Control record as: $fieldlist={"reason", "risk", "planned.start"} In the Subroutine call, specify the Value input as: Name: names Value: $fieldlist **Note**: You can use any variable name you choose, however, you are responsible for ensuring that your variable name doesn't corrupt a variable used by the application validate.fields.
text	current.format() or <form name>	Executes a specific list of validity table definitions defined for the form just displayed to the user.

5. Enter a message in the **Error Message** field.
   The recommended error message is: The original record is displayed.
   If a message is undefined, an unfriendly error message is issued when returning to the data record.
6. Click **Save**.

Related topics

Invoking validity table processing
Data validation
Format Control
Format Control processes
Displayoptions

Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Create a validity lookup option in Format Control
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.15. Create a validity lookup option in Format Control

Access the Format Control record for the form in which you want to invoke validity table processing. If a record doesn't already exist, you must create one. For information on how to access and use Format Control, see the related topics.

1. Click **Tailoring** > **Format Control**.
2. Type a **Name** and then click **Search**.
   Service Manager opens a list of Format Control records.
3. Click the record you want to update to open it.
4. Click **More** or the More Actions icon and choose **Additional Options**, or click **Addl Options** in the form.
5. Enter the following field values:

Field	Value
Condition for Option	true
Option	Validity Lookup
Application	Application validate.fields
Error Message	The original record is displayed. **Note**: This is the recommended message. If this field is left blank, you will receive an unfriendly error message when you return to the data record.
Reset on Return	true

6. Enter the applicable parameters for **validate.fields** in the **Name** and **Value** fields using the information from the following table:

Field	Value
name	cursor.field.name()
second.file	$file
cond.input	val("true", 4) **Note**: The data passed to the Boolean parameter must be a type 4 (hence the VAL() statement for the Value). As an alternative, a **$variable** could be defined in the *initialization expressions* (for example $flag=true) and then passed to the subroutine.

7. Click **Add** to add a new Format Control record.
8. Click **Save** to save your updates.

Related topics

Data validation
Format Control processes
Format Control
Accessing Format Control
Using expressions in Format Control
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Call validate.fields from Format Control
Validity validation rules

1.2.4.8.16. Call validate.fields from Format Control

1. Click **Tailoring** > **Format Control**.
2. In the **Name** field, type the name of the form for which you want to invoke validity table processing.
3. Click **Search**.
4. If a Format Control record doesn't exist for your form, create one. Click **New**.
5. Click **Subroutines** in the form, or select it from the More Actions menu.
6. Enter the appropriate parameters for **validate.fields** in the **Name** and **Value** fields using the information from the following table:

Field	Value
name	cursor.field.name()
boolean1	false
second.file	$current.file.variable
cond.input	true

7. Click **Save** to save changes to an existing record.
   Caution: If you are attempting to add a new record from an existing record, make sure that you don't click Save because doing so will replace the existing record with the new record. If you are adding a new record, click Add.
8. Click **Add** to add a new record.

Related topics

Data validation
Format Control processes
Displayoptions
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Validity validation rules

1.2.4.9. Validity validation rules

- Each entry must have at least one entry in the **Files/Formats** field.
- Each entry in the **Files/Formats** field must be a valid database dictionary name or form name.
- The **Field Name** must be defined in each database dictionary listed in the **Files/Formats** array or to the database dictionary associated with the form.
- The data type of the **Field Name** must be the same for each file listed in the **Files/Formats** array or for the file associated with the form.
- The **Unique ID** is a user-provided value that uniquely identifies the record.
- The unique key of the validity table contains the following fields: **field.name**, **sequence**, and **filename**. Each combination of the Field Name, Sequence, and each element of the Files/Formats array in the validity record must also be unique within the validity table. This check is performed before the record is added or updated. If duplicates are found, the add/update action is not performed, and the following message is displayed: This record contains an invalid duplicate key.
- The value in the **Alt Application** field must be a valid RAD application.
- The **Secondary File Query Filename** value must be a valid database dictionary name.
- The **Lookup Fields** must be defined to the **Secondary Query Filename** and their data types must be the same as the Field Name.
- The **Val Fields** must be defined to the **Files/Format** file and their data types must be the same as the Field Name.
- The Record list format must be a valid form name.

- The value in the **No Recs Option** field must be the word **Proceed**, or **Return**. If neither is selected the system defaults to Proceed when the record is validated.

Related topics

Data validation
Look-up processing
Display value summary details
Display range summary details
Create validity table definitions
Add field-level definitions
Add value definitions
Add range definitions
Validate the validity definitions
Delete an entire validity record
Delete a single value or range definition
Delete value or range definitions from a table
Print a detailed report of an entire validity record
Validate fields during record processing
Create a validity lookup option in Format Control
Call validate.fields from Format Control

1.2.5. Global lists

The global initer and global lists work together to generate lists for the Service Manager system.

Global lists are stored in the system and are available to any application. Use the global lists tool to add a combo box to a form that displays a list of values, based on a table in your database. For example, set your combo box **DisplayList** value to *$G.time.zones*, and the system displays a drop-down list of time zones from the *tzfile* file.

The global initer builds lists of records from the database. Generally, these lists are used to fill the drop-down lists in combo boxes on display forms. Examples of common lists include a list of all operators in the system, all Incident categories in the system, or all assignment groups on file.

The Startup Lists global list stores all the global lists where the build.startup field equals true. The global list is in the listrepository file. During logon, the system checks the listrepository entry and builds all global lists in this file. If the global list file changes, the Startup Lists global list is marked as expired and rebuilds the next time the lister background process runs. This updates the listrepository record.

Note: Since the lister background process doesn't run continually, there may be a slight delay between the time a global list changes and when it appears in the listrepository record.

The global initer consists of two parts:

- A server side element which generates and packages lists according to a user-defined schedule.
- A client-side requester which queries lists from the server and places them into client-side memory.

Lister

The background scheduler (lister) wakes up every sixty seconds to rebuild expired lists, which are then stored in the record.

Note: Lister variables can be defined for *scalar* type fields only and not for array fields.

Global initer considerations

- For performance considerations of the Service Manager server, a global list should contain no more than 2800 individual elements. Exceeding this limit may degrade the server performance.
- The total size of the **globallist** record, which contains the full lists, can't exceed the logical file limit set for the system.
- The sort field must be a key on the table which is used to generate the list.
- No matter what you set as the reset time, the server initer can only refresh the lists once every time its scheduler wakes up. Thus, if you have a list with reset time = 5 seconds and the scheduler runs every 60 seconds then the list will actually refresh every 60 seconds.

Returning a list to your client

Use one of the following methods to return a particular list, or set of lists, to an Service Manager client:

- Build a list on startup.
- Configure the lists with Format Control.

Configuring lists with Format Control

Use Format Control to ask for a list only when a specific form is displayed. This procedure is most efficient when the form defined is not displayed frequently.

Access a list record

1. Click **Tailoring** > **Tailoring Tools** > **Global Lists**.
2. Enter optional search criteria, and then click **Search**.
   A global list of definitions opens.
3. To create a new list, click **Add**.
4. If you edit an existing list, click **Save**.
5. Click **OK**.

Bind a list to a form

Lists are bound to the forms they serve. The global variables are applied in Forms Designer.

1. Click **Tailoring** > **Forms Designer**.
2. Type the name of the form you wish to modify in the **Form** field of the Forms Designer dialog box.
   For example, type: `contact.detail.subform.` This is the name of the subform used by `problem.template.open.g`, `problem.template.update.g`, and `problem.template.close.g` to display contact information.
3. Click **Search**.
   The contact detail subform opens.
4. Click **Design**.
5. Add a combo box with a label called **Time Zones** to the form, using the following values:

Field	Value
Input	time.zones
ValueList	$G.time.zones
DisplayList	$G.time.zones

6. Wait for lister to refresh the time.zones global list, or go to the list definition panel (**Tailoring** > **Tailoring Tools** > **Global Lists**) and reset the refresh time.
   Note: If you leave the DisplayList value blank, the contents of the ValueList will be used for both lists.

Build lists on startup

The **Build List on Startup** option, if enabled for a global list, doesn't initiate a "rebuild" of the list at user login, but simply makes the current list available to a new logged-in user.



Note

- Prior to version Service Manager 9.41, all global lists with the **Build List on Startup** option enabled were loaded at user login by applications, which might cause significant login delays. As of version 9.41, this functionality is optimized so that these global lists aren't loaded into memory unless they are needed.
- For a newly added global list, a restart of the Service Manager Server is required unless the name of the list variable starts with "$G.gl.".

If a global list has a "Filename" and "Limiting SQL" defined, to automatically rebuild the list when something is changed in the source file, the **lister** background process must be running.

A global list is rebuilt on either of these actions:

- A user clicks the **Rebuild Global List** option (the current and new user sessions get the new list)
- The **lister** process is running against this global list (whose expiration time must be in the past)



Caution

Caution If you have created a Global List variable without the **Build List on Startup** option enabled, you need to manually rebuild the contents of the variable to make the variable ready for use. For example, if you create a global list $G.test without selecting the **Build List on Startup** option and you don't rebuild it manually, when you go to the RAD Debugger and type d $G.test, the RAD Debugger returns a NULL value.



Tip

Tip To avoid memory problems, you are recommended to enable the **Build List on Startup** option only for global lists that have less than 250 items.

Use the following procedure to set the **globallists** record to build on startup.

1. Click **Tailoring** > **Tailoring Tools** > **Global Lists**.
2. In the List Name field, type the name of the list you want to view. For this example, type **time.zones**.
3. Click **Search**.
   The time.zones file opens.
4. Select **Build List on Startup?**.
5. Click **Save**.
6. Open an incident record to view the new field.

Configure lists with Format Control

1. Determine the base name of the form, which you added to your list.
2. Click **Tailoring** > **Format Control**.
3. Type the *base* name of the form in the **Name** field.
   For this example, create a new record for the form `problem.template.update`.
   Note: If you add a combo box to a subform, attach the Format Control record to any base form using that subform. (Do not attach the Format Control record to the subform itself.)
4. Click **Search**.
5. If a Format Control record doesn't exist for your form, create one. Click **New**.
6. Click **Subroutines** in the form, or select it from the More Actions menu.
7. Select **Show Expanded Form** from the More Actions menu.
8. Scroll to the first empty section and set up a Subroutine call using the following values:

Parameter	Value
Application name	apm.global.initer
Names/Values	*See next table*
Before	true
Display	true

Names	Values	Definition
names	{"time.zones"}	Array of lists to be moved
name	create	Action to perform (create lists)

9. Click **Save** to save changes to an existing record.
   Caution: If you are attempting to add a new record from an existing record, make sure that you don't click Save because doing so will replace the existing record with the new record. If you are adding a new record, click Add.
10. Click **Add** to add a new record.

This Format Control record returns the **time.zones** list to all incident records displaying the Contact Detail tab, regardless of mode.

Calling the **apm.global.initer** several times from Format Control will not slow the system appreciably. The system automatically keeps track of what lists are already on the client and will not recover them unnecessarily; however, if you want the Global Initer to reassemble the list regardless, invoke it with the refresh parameter (rather than the create value listed in the above). called with the refresh parameter, the Global Initer returns all the lists passed to the names array, even if they are already in client memory.

Determine if a scheduler record exists

1. Click **Tailoring** > **Database Manager**.
2. Type schedule in the **Form** field of the Database Manager dialog box.
3. Press **Enter** or click **Search**.
4. Open schedule from the record list (double-click the record or click Enter). A blank Schedule File record is displayed.
5. Type List Runner in the **Name** field.
6. Click **Search**.

If a record is found, it will open for viewing and editing.

Determine if the regen cycle is realistic

1. Click **Tailoring** > **Tailoring Tools** > **Global Lists**.
2. Click **Search**.
   A blank record from the **globallists** file is displayed.
3. Click **Search**.
4. Select the list you want to regenerate from the Record list.
5. Check the value in the **Regen Every** field.

Use a short time interval for lists that change frequently and a longer interval for lists that change only occasionally.

Determine when a list was last regenerated

1. Click **Tailoring** > **Tailoring Tools** > **Global Lists**.
2. Click **Search**.
   A blank record from the globallists file opens.
3. Click **Search**.
4. Select the list you want to regenerate from the Record list displayed.
5. Check the value in the **Expiration** field to see when the list was last regenerated.

Move a Global List to a client

To move a global list into client side memory, you call a routine called **apm.global.initer**.

This routine accepts two parameters:

- An array containing the name of the lists that you want.
- A keyword, either create or refresh.

When **apm.global.initer** is invoked with the refresh parameter, it collects the newest version of its list from the server, whether that list already exists in the client's memory space. If invoked with the create parameter, it collects a list from the server only if a copy doesn't already exist.

If you are programming in RAD, you can invoke **apm.global.initer** directly from your code. If you are using the Global Initer to fill combo boxes for display on a form, you can invoke the Global Initer as a subroutine using Format Control for the format in question.

Regenerate all lists

If none of the troubleshooting steps to this point have succeeded in regenerating your list, you should regenerate all the lists in the system. This procedure makes all the lists in the system obsolete and enables you to update the entire file.

1. Click **Tailoring** > **Database Manager**.
2. Select **Administration Mode**.
3. Type **apm.global.list.entry** in the Form field.
4. Click **Search**.
   A blank search form from the globallists file opens.
5. Click **Search** to display a list of lists.
6. Click **Mass Update**.
7. Click **Simple Update**.
   A blank form opens. This form is identical in appearance to the *lister* record, but contains different option buttons.
8. In the **Expiration** field, change the date to any date in the past. You can either type the date and time, or select a date from the calendar.
9. Click **Execute**.
   The expiration date of all the lists in the globallists file is reset.
10. Return to the home menu.
11. Type `*aapm.server.initer` in the command line.
12. Press **Enter**.
13. Log off Service Manager, and then log back on.
    All the lists in the system are regenerated whether they are actually obsolete.

Regenerate obsolete lists

If you perform all the troubleshooting steps and the system still doesn't display the most current version of your list, regenerate all the obsolete lists in the system. For a large system with numerous obsolete lists, this procedure may take a few minutes.

You can change the expiration date of a list to a date in the past prior to regenerating, in order to force regeneration.

Note: If the expiration date of your list has not passed, the list will not be flagged as obsolete, and therefore not regenerated.

To regenerate obsolete lists:

1. Type `*aapm.server.initer` in the command line.
   Running this application will force the system to regenerate all obsolete lists.
2. Log off Service Manager, and then log back on.

Start the server side component of the global initer

The server side component of the Global Initer consists of a routine called apm.server.initer. A background scheduler called lister has been defined to run apm.server.initer.

If lister did not start with the system:

1. Click **System Status** on the System Navigator.
   The System Status utility opens.
2. Click **Start Scheduler**.
3. Double-click **lister.startup**.
   Every sixty seconds, lister checks and refreshes any global lists that have passed their reset time.

See a complete list of all the global lists on my system

1. Click **Tailoring** > **Tailoring Tools** > **Global Lists**.
2. Click **Search**.
   A blank search form from the *globallists* file is displayed.
3. Click **Search**.
   A list of all the global lists opens. You can scroll through the complete list of records.

Verify lister status and configuration

1. Click **System Status** in the System Navigator.
2. Check to see if lister appears in the list of processes running on your system.
3. If lister in not on the list, click **Start Scheduler**.
4. Double-click **lister.startup** in the Startup Record list to start the program.
   Note: Double-click **startup** to start all processes.

View changes to a list

1. Click **Tailoring** > **Tailoring Tools** > **Global Lists**.
2. Click **Search**.
   A blank record from the *globallists* file is displayed.
3. Click **Search**.
4. Select the list you want to view from the Record list.
5. Check that the changes appear.

1.2.6. Record tag localization utility

The record tag localization utility allows for the localization of fields used to identify records in a given table. Users may use this feature to translate such tags so that they display in their appropriate translations when a user logs into a localized version of Service Manager. This allows the fields to appear in the language of the session associated with the user rather than in their true saved form

Related topics

Localization process
Codes
Message maintenance with record tag localization

1.2.6.1. Localization process

Before you begin the localization process, make sure that all desired languages are activated in the language table.

The localization process consists of four steps:

- Enabling the table for tag localization by modifying the corresponding data policy record.
- Updating all corresponding message records to contain the correct translations.
- Creating global lists for the table that utilize these message records.
- Invoking the localization utility to define the fields that will use these new global list values.

Related topics

Accessing and localizing message records for a codes-based global list
Invoking the localization utility
Enable a table for tag localization
Create localized global lists

1.2.6.1.1. Enable a table for tag localization

To enable a table for localization:

1. Access the data policy record for the table.
2. On the General tab, check the Localized Table checkbox, which then makes the Localized Tag Field available.

3. Set Localized Tag Field to the name of the field whose values you wish to localize. This provides the localized record tag.
4. Save your changes.

For example: Use the following inputs to enable localization for the name field in the category table.

Field Name	Value
Localized Table	checked
Localized Tag Field	name

Wait for the lister process to rebuild the localizedTables global list. When this is done, view the global list to ensure that it includes the table you enabled for tag localization.

Related topics

Localization process
Data Policy
Global lists
Accessing and localizing message records for a codes-based global list
Invoking the localization utility
Access Data Policy
Create localized global lists

1.2.6.1.2. Access and localize message records

Once localization is enabled, message records for each active language are created for every existing record in the table. If English and French are enabled for your system, then English and French message records exist for every category record once the category table is marked for localization. These messages contain the following values:

- **Class:** local:<TABLE>, where <TABLE> refers to the name of the table.
- **Message ID:** <UNIQUE>, this refers to the values that make up the unique key of the record.
- **Text:** <LOCALIZED VALUE>, this refers to the value of the localized field. This is the value that needs to be modified to contain the correct translation.

To access and localize these message records:

1. Access the table itself and bring up a record.
2. Click **More** or the More Actions icon and then select View Localized Record Tags. This lists every message that corresponds to the record.
   Note: The View Localized Record Tags action is available only for enabled tables that utilize the db.view displayscreen. To make this option available for a table that uses a different displayscreen, you need to create a similar displayoption record for that displayscreen.
3. Drill into each message record and modify the Text field so that it contains the correct translation for the localized field.
4. Repeat these steps for the remaining records in the table.

Related topics

Localization process
Accessing and localizing message records for a codes-based global list
Message maintenance with record tag localization
Enable a table for tag localization
Create localized global lists
Invoking the localization utility

1.2.6.1.3. Create localized global lists

The record tag localization utility makes it possible to localize a query-based list. You can use this utility to localize lists that are defined by a query.

To localize a query-based global list:

Note: You must sign out and then sign in again for the global list change to take effect.

1. Create or access an existing global list record that's based on the Limiting SQL field.
2. Check **Use Localized Values?**.
3. Ensure that List Variable and Display Variable are set to different variable names.
4. Save your changes.
5. Wait for the lister process to build the list.

Example: use the following inputs for Global List Definition:categories.active to return a list of all active categories.

Field Label	Value
List Name	categories.active
Build List on Startup?	checked
List Variable	$G.categories.active
Display Variable	$G.categories.active.local
Limiting SQL	active=true or active=NULL
Use Localized Values?	checked

After the list is built, the localized lists for all active languages display in the table under the Value List field on the Global List Definition form.

These lists are based on the contents of the message records that were updated in Accessing localizing message records (Accessing and localizing message records for a codes-based global list). The display variable will be bound to the display list for the language associated with the user session.

Related topics

Localization process
Global lists
Accessing and localizing message records for a codes-based global list
Creating localized codes-based global lists
Enable a table for tag localization
Associate a field with a codes-based global list

1.2.6.1.4. Invoking the localization utility

Once the global lists are defined and built, you may invoke the localization utility to define the format fields to utilize the global lists.

Note: To access the localization utility, click **Tailoring** > **Localization Utility**.

The localization utility is comprised of the following fields:

- **Table Name**: the table containing the fields you wish to localize.
- **Field Name**: a list of fields to localize.
- **Global List**: a list of global lists to be bound to those fields.
- **Formats**: a list of formats on which to apply each field-global list association.

Example: Use the following inputs to bind the Category (*category*) field in the probsummary table to the *categories.active* global list created in Create localized global lists. This example is set up to take effect on the *IM.update.incident* format.

Field Label	Value
Table Name	probsummary
Field Name	Category
Global List	categories.active
Formats	IM.update.incident

After defining your requirements, you must save the record and click **More** or the More Actions icon and then select Run Localization Utility to run the utility.

The localization utility:

- Updates the corresponding data policy record so that each specified field is set to the associated global list in the Global List field.
- Updates all the listed formats containing these fields to use the display list values of the associated global list.

The Messages pane of the localization utility screen displays a list of the updated data policy record and all updated formats.

Note: Once the formats are updated by the localization utility, removing them from the Formats list and rerunning the utility does **not** reverse the changes. The utility is designed to be a one-way process so any reversions need to be made manually inside Forms Designer. Likewise, all data policy changes are **not** reversed and will need to be reverted manually.

Once the localization process completes, all the defined fields should appear localized when displayed using one of the converted formats. Instead of English, they display in the language associated with the user session.

Related topics

Localization process
Global lists
Accessing and localizing message records for a codes-based global list
Enable a table for tag localization
Create localized global lists
Associate a field with a codes-based global list

1.2.6.2. Codes

The codes table is a repository for storing user-defined values that you wish to localize. Enabled for localization, codes leverage the record tag localization utility and may be used for localizing lists of codes or status values. It offers an alternative to defining localized user-defined global lists without the difficulties of having to hardcode and to maintain the values inside the global list definition record.

To access codes, click **Tailoring** > **Codes**.

Codes are comprised of the following fields:

- **Type**: a name by which to classify a set of values.
- **Value**: a value to be localized.
- **Sort Order**: a number indicating the sort order of this value.

Related topics

Defining a set of values for a codes-based global list
Accessing and localizing message records for a codes-based global list
Creating localized codes-based global lists
Associate a field with a codes-based global list

1.2.6.2.1. Defining a set of values for a codes-based global list

When defining a set of values, a record needs to be created for each value and must be assigned to the same type.

For example: Use the Codes function to create (define) a series of codes for the Boolean values true and false.

Type	Value
Boolean	false
Boolean	true

Once all the codes are defined, proceed to Accessing and localizing message records for a codes-based global list of the localization process.

Related topics

Accessing and localizing message records for a codes-based global list
Creating localized codes-based global lists
Codes
Associate a field with a codes-based global list

1.2.6.2.2. Accessing and localizing message records for a codes-based global list

As with any table that's enabled for localization, the corresponding messages may be accessed from each codes record by clicking **More** or the More Actions icon and then selecting the View Localized Record Tags.

Once all the messages are updated to contain the correct translations, proceed with Creating localized codes-based global lists.

Related topics

Creating localized codes-based global lists
Codes
Message maintenance with record tag localization
Defining a set of values for a codes-based global list
Invoking the localization utility

1.2.6.2.3. Creating localized codes-based global lists

To define a global list for a set of codes, the global list definition must contain the following values:

- **List Field**: value
- **Filename**: codes
- **Limiting SQL**: type="<TYPE>", where <TYPE> refers to the type of codes you wish to localize.

In addition, you must:

- Check the Use Localized Values? checkbox.
- Ensure the List Variable and Display Variable fields are set to different variable names.
- Wait for the lister process to build the list.

Example: use the following inputs for Global List Definition:Boolean Values to create a localized list of Boolean Values.

Note: that the example references records that were created in the example found in Defining a set of values for a codes-based global list.

Field Label	Value
List Name	Boolean Values
Build List on Startup?	checked
List Variable	$G.booleans
Display Variable	$G.booleans.local
Limiting SQL	type="Boolean"
Use Localized Values?	checked

Related topics

Defining a set of values for a codes-based global list
Codes
Global lists
Accessing and localizing message records for a codes-based global list
Associate a field with a codes-based global list
Create localized global lists

1.2.6.2.4. Associate a field with a codes-based global list

To associate a field with a codes-based global list, you map the field to the global list in the same way you would with any other list.

Example: In the localization utility, use the following inputs to bind the Visible to Customer (cust.visible) field to the Boolean Values global list. It will take effect on the probsummary.qbe.g format.

Field	value
Table Name	probsummary
Formats	probsummary.qbe.g

Use the following values for the Field Name and Global List entries.

Field Name	Global List
Visible to Customer	Boolean Values

As a result of this operation, when you display a list of incidents, the column values for cust.visible appear localized if the probsummary.qbe.g format contains that field.

Related topics

Codes
Defining a set of values for a codes-based global list
Accessing and localizing message records for a codes-based global list
Creating localized codes-based global lists Create localized global lists
Enable a table for tag localization

1.2.6.3. Message maintenance with record tag localization

The messages utilized by the localization process are maintained in the following manner:

- If a record is added to the localized table, corresponding message records are created for it.
- If a record in the localized table is updated so that the localized field is changed, its message records are deleted and recreated to reflect this. The Message ID of the new message records contains the new unique ID and the Text field is set to the new localized tag field value.
- If a record is deleted from the localized table, its message records are removed from the system.
- If the data policy for a localized table is deleted, all corresponding message records are removed from the system.
- If an enabled table is then disabled for localization, all corresponding message records are removed from the system.

Update message records for activated or deactivated languages

If a language is made active, new message records don't get created for it. However, rather than having to create these records from scratch you can perform the following steps.

To update messages for activated languages:

1. Access the data policy record for each localized table.

2. Click **More** or the More Actions icon and then select Update for Active Languages.
   This creates message records for every active language that don't already exist.
3. Ensure that the new messages are properly translated.
4. Rebuild all applicable global lists to reflect the new changes.

Any languages that are deactivated or activated after launching the localization process aren't automatically accounted for. If a language is made inactive, none of the corresponding message records are removed. The records are left in the system in case the language is reactivated in the future. Hence, they will need to be manually deleted if desired.

Related topics

Record tag localization utility
Localization process

1.2.7. Help systems

Service Manager has two Help systems:

- The online Help system explains Service Manager concepts and how to use Service Manager features. When you want to learn more about a feature or how to complete a task, online Help explains what you need to know.
- The field Help system contains a help on the fields in a form. When you are filling a form with data, field Help can tell you what information to input in any field.



Note

Note In the web client, the **disableKeyHelp** parameter controls the field Help system.

Related topics

Online Help system
Field help editor
Creating field help
Preparing to create field help
Detect the help record to display
Add or edit help records
Access the help table
Review field help records

1.2.7.1. Online Help system

Service Manager has a web-based online help system that you can't customize. The Help system describes features, concepts, resource information, and steps that you can follow to perform Service Manager tasks.

Related topics

Help systems
Field help editor

1.2.7.2. Field help editor

The field Help system contains a help on the fields in a form. When you are filling a form with data, field Help can tell you what information to provide in any field.

The Field help editor enables you to add and edit help for any Service Manager field, form, table, term, or topic. Using it you can add a unique help record that describes the data in any field.

Service Manager stores field help as records in the `help` table. The table has fields that enable you to specify the brief description, detailed description, table name, form name, field name, related information, term and topic for a help record.

You can add, edit, or delete individual help records through the Field help editor. Alternatively, since the field help is stored in a standard Service Manager table, you can use the standard table related functions to add, edit, or delete batches of help records, including import, export, load, unload, and mass add, update, and delete.

You can run queries to find records in the help table, just as you would on any other Service Manager table, using either the standard QBE search, or the Advanced search functions.

Related topics

Creating field help
Preparing to create field help
Detect the help record to display
Online Help system
Add or edit help records
Access the help table
Review field help records

1.2.7.2.1. Creating field help

When creating field help, keep in mind that the forms you see as you step through a process may not be the ones the user sees. What you are looking at is, most likely, not the only place that displays the field.

- The forms displayed vary with the choices the user makes.
- The fields displayed on the forms vary with the choices the user makes and with the user's permissions.
- The use of the field varies depending on user permissions and where the field displays. (For example, if data gets entered into read-only fields.)

When creating field help, you may need to do any of these tasks:

- Access the help table.
- Add or edit field help records.
- Run a query through the help table.
- Export a batch of existing field help records and edit them.
- Import a batch of new or edited field help records.
- Perform a mass update on a list of help records.
- Unload a batch of field help records from your local machine and load them into the development machine.
- Review new or updated field help records.

Related topics

Preparing to create field help
Field help editor
Detect the help record to display
Online Help system
Add or edit help records
Access the help table
Review field help records

1.2.7.2.2. Preparing to create field help

To create field help efficiently, make these changes in your local Service Manager system.

- Show the context-sensitive help debug information.
- Activate the command line for the System Administrator (or the operator you want to enable to create field help) on your local server.
- Set up the columns displaying the help record list.
- Turn on the Administration perspective.

Show context-sensitive help debug information

1. Start the Service Manager client.
2. Sign in or close the Connections dialog box.
3. Click **Window** > **Preferences**.
   The Preferences dialog box opens.
4. Click **Service Manager** > **Appearance**.
   The Appearance dialog box opens.
5. Select **Show context-sensitive help debug information**.

6. Click **Apply**.
7. Click **OK**.

Exit the Service Manager client, and then restart it.

Service Manager now displays the file (table), form, and field information when you press Ctrl+H for a field.



Note

Note

- You can use such field help information to add or edit field help records or identify field names to be used in stored queries.
- The Web client uses the `viewcontexthelp=true` URL parameter to display field help information.

Activate the command line

1. Start the Service Manager client.
2. Sin in to your local server as a System Administrator.
3. Type `operator` on the command line and click **Execute Command**.
   Service Manager displays the operator form (operator.g).
4. Type `System.Admin` as the sing-in name (or specify the operator you want to set the permissions for), and then click **Search**.
5. Open the Startup tab and select **Activate Command Line at Startup**.
6. Click **Save**.
7. Click **Ok** to exit the record.
8. Click **Back** to exit the form.
9. Exit the Service Manager client, and then restart it.
   Service Manager now displays the Command Line to that operator.

Set up the columns displaying the help record list

1. Open the help table in the Help Editor and click **Search** to display the help records.
2. Click **More** or the More Actions icon.
3. Click **Modify Columns**.
   Service Manager displays a form that enables you to edit the columns in the record list.
4. Type or select the following, one per line in this order:
   - topic
   - file.name
   - format.name
   - field.name
   - term
   - brief
   - detail
   - Pending Review
   - sysmodtime
5. Click **Proceed**.

Service Manager displays the record list with the columns you selected.

Turn on the Administration perspective

You can turn on the Administration perspective by doing one of the following:

1. Click **Window** > **Open Perspective** > **Other**, and then select **Administration**.
2. In the Service Manager System Navigator on the left side of the window, click **Administration Perspective**.

Service Manager now displays Administration tabs at the bottom of your forms.

Related topics

Creating field help
Field help editor
Detect the help record to display
Online Help system
Add or edit help records
Access the help table
Review field help records

1.2.7.2.3. Access the help table

Use one of the following procedures to access the help table:

- Type `help` on the Service Manager command line and press Enter.
- Use Database Manager.
  Click **Tailoring** > **Database Manager**. Type `help` in the Form field, and then click **Search**. Service Manager displays the Help Description tab of the help form (help.g). Use this form to add or edit help records.

Note: If you don't see the command line, you must activate it for the operator you are using.

Related topics

Creating field help
Preparing to create field help
Field help editor
Detect the help record to display
Online Help system
Add or edit help records
Review field help records

1.2.7.2.4. Check the fields that a form contains

1. Make sure you have the **Administration Perspective** open. To do this, do one of the following:
   - Click **Window** > **Open Perspective** > **Other**, and then select **Administration**.
   - In the Service Manager System Navigator on the left side of the window, click **Administration Perspective**. This turns on the Administration Perspective, which allows you to see the administration tabs at the bottom of the form.
2. Open the form using Database Manager.
   Note: Don't use the System Navigator as it will open the form in Forms Designer, which won't give you the data you need.
3. Select the **Detail Data** tab.
   The Detail Data tab contains the XML that generates the form.

The sample XML below describes two fields:

- **Contact for this interaction**: A comfill for the `callback.contact` field.
- **Notify By**: A selection list for the `callback.type` field, containing the options None, Email, Telephone, and other.

```
<label x="2" scType="Label" id="Label13" width="32" height="2" forecolor="0" y="2">Contact for this
interaction:</label>
<comfill ref="instance/callback.contact" x="36" scType="ComFill" id="comfill1109271071725" mandatory="1"
idFill="9" width="36" idFind="8" arraylength="1" tabindex="1"
foreignkeyref="Links/LinkLine[@SourceField='callback.contact']" scComboButtonVisible="0" height="2"
y="2" dataType="string" />
<label x="2" scType="Label" id="" width="32" height="2" forecolor="0" y="6">Notify By:</label>
<select ref="instance/callback.type" BoxLines="4" x="36" id="" mandatory="1" width="36" arraylength="1"
selectonly="1" height="2" y="6" dataType="string">
<option value="None">None</option>
<option value="Email">Email</option>
<option value="Telephone">Telephone</option>
<option value="Other">Other</option>
</select>
```

Note: You can copy and paste the data from this form into another file, and edit it down to something easier to read.

Related topics

- Creating field help
- Preparing to create field help
- Field help editor
- Detect the help record to display
- Add or edit help records
- Access the help table
- Check the fields that a table contains
- Check whether help exists for a field

1.2.7.2.5. Check the fields that a table contains

The System Navigator displays the fields in a table, along with many other table features.

To see the fields in the System Navigator:

1. Click **System Definition** > **Tables** > ***tablename***.
2. Select **Fields** to see fields.
3. Select **keys** to see the keys.

Related topics

- Creating field help
- Preparing to create field help
- Field help editor
- Detect the help record to display
- Add or edit help records
- Access the help table
- Check the fields that a form contains
- Check whether help exists for a field

1.2.7.2.6. Check whether help exists for a field

Service Manager may not display help that's available for a field when you press **Ctrl+H** in the Windows client or press **F1** in the Web client.
To check the complete list of help records for a field, if there are any, you must search the help table.

1. Type `help` in the Service Manager command line and press Enter.
2. Specify your search criteria.
    - To search for help on a field in a table, specify the **Field Name** and the **Table Name**.
    - To search for help on all fields in a table, specify the **Table Name**.

    - To search for help on a field in a form, or if you don't know the table, specify the **Field Name**.
3. Click **Search** or press Enter.

Service Manager returns records that match your query, if there are any.

Related topics

- Creating field help
- Preparing to create field help
- Field help editor
- Detect the help record to display
- Add or edit help records
- Access the help table
- Check the fields that a table contains
- Check the fields that a form contains
- Review field help records

1.2.7.2.7. Detect the help record to display

A field help record must have a field name and either a table name or a form name for display.

- The default record for a field must have the field name and the table name, but no form name. Service Manager displays the default help record for a field for all forms that use the field, excluding those that have a form specific help record.
- A form specific help record for a field includes the form name. Service Manager only displays that particular help record when it displays that particular form.

If there is no help record for a field, Service Manager displays **No Description**.

If there is no form name or table name in a help record, Service Manager will not display that record as help on a field. However, you can still search for the record and open it using the Field help editor. This is useful if you want to create a glossary of terms, for example.

Best practice

To ensure consistent field descriptions on all forms, omit the form name from the field help record.

If you add a form name to a field help record, the text displays only when Service Manager displays that form. Adding a form name can be a useful technique for documenting special forms, but may increase the number of help records to maintain.

Related topics

Creating field help
Preparing to create field help
Field help editor
Add or edit help records
Access the help table

1.2.7.2.8. Add or edit help records

The user can also click "Accept" instead of Add/Save after editing the help. This will immediately publish the help topic with no additional review.

1. Click **Tailoring** > **Database Manager**.
2. Type **help** in the Form field, and then click **Search**.
3. Double-click the `help` (help.g) form.
   Service Manager displays the Help Description tab of the help form (help.g). Use this form to add or edit help records.
4. Type a **Brief Description**.
5. Type the **Detailed Help** information, if necessary.
6. Select the **Keywords** tab.

7. Specify the **Field Name**.
8. Specify the **Table Name**.
9. (Optional) You can add the following in the Keywords tab:
   • Specify the **Form Name** - Use this field to create form-specific help.
   • Specify **Related Information** - Use this field to create a link to field help for related fields.
   • Specify the **Term** - Use this field to define glossary terms.
10. (Optional) You can specify the **Topic** in the Help Description tab. Use this field to input help on specific areas of Service Manager.
11. Click **Add** or **Save** to create new or revise existing field help.
    The help topic is now marked **Pending Review**. The help topic is not published until it is reviewed and accepted. For more information, see Review field help records.

> **Tip**
> Tip You can click **Accept** instead to immediately publish the help topic without additional review.

Related topics

Creating field help
Preparing to create field help
Field help editor
Detect the help record to display
Online Help system
Access the help table
Review field help records

1.2.7.2.9. Review field help records

New or revised help topics must be reviewed before they can be published. If you don't complete this step, the help records will not be visible to users.

To review field help records, follow these steps:

1. Connect to the server that contains the help you want to review.
2. Run this query against the help table.
   `pending.review="true"`
   Service Manager displays the records that currently need review.
3. Use one of the following procedures:
   - Click **Accept** to indicate that you have reviewed an individual record.
   - Use the following Mass Update to indicate that you have reviewed the list of records.
     `pending.review in $file=false`

> **Note**
> Note If more than one person has input field help, you will need to refine your query to find the specific records you want to review.

1.2.8. Sequential number setup

The Sequential Number file is used in conjunction with Format Control to generate sequence numbers for records in a database. The sequence number is automatically incremented or decremented when a new record is added. For instructions on creating a Format Control record with sequential numbering, refer to Format Control Help.

Related topics

Sequential number file
Access the sequential number file
Create a simple number counter
Use decrement in sequential numbers

Use prefix and suffix in sequential numbers
Update a sequential number record
Delete a sequential number record

1.2.8.1. Sequential number file

The Subroutines process of Format Control creates a Sequential Number File record when the application getnumb.fc is called. The parameters passed to the application from within Subroutines define the values in the number file fields. These values determine how the sequential number opens and what other information it contains. You can define number file values either directly in the number file or via parameters passed from Format Control. However, if you use the number file to change values also defined in the Format Control subroutines process, they are overridden by the Format Control values.

Related topics

Sequential number setup
Access the sequential number file
Create a simple number counter
Use decrement in sequential numbers
Use prefix and suffix in sequential numbers
Update a sequential number record
Delete a sequential number record
Service Manager Programming

1.2.8.2. Access the sequential number file

The Sequential Number file is used in conjunction with Format Control to generate sequence numbers for records in a database.

To access the sequential number file:

Click **Tailoring** > **Tailoring Tools** > **Sequential Numbers**.

The sequential number file opens.

Related topics

Sequential number file
Sequential number setup
Create a simple number counter
Use decrement in sequential numbers
Use prefix and suffix in sequential numbers
Update a sequential number record
Delete a sequential number record

1.2.8.3. Create a simple number counter

The following example illustrates how to create a record called employee in the Sequential Number File to automatically increment employee numbers starting with 1.

1. Access the Sequential number file.
2. Create a new file. Type `employee` in the Class field for this example.
3. Type `0` in the Last Number field to begin incrementing from zero.
4. Type a short description for the number class.
   For example: Employee ID number counter.
5. Type `1000` in the Reset Point field.
6. Type `1` for the Increment/Decrement field value.
   Since the Decrement field was left blank, each number will increase by one.
7. Click **Add**.

The new sequential number record is added to the sequential number file.

Related topics

Sequential number file
Sequential number setup
Access the sequential number file
Use decrement in sequential numbers
Use prefix and suffix in sequential numbers
Update a sequential number record
Delete a sequential number record

1.2.8.4. Use decrement in sequential numbers

You can use sequential numbers to decrement a starting value. For example, you can decrement a quantity field when deleting stock from inventory. This example shows you how to decrement a value starting at 1000.

1. For this example, type `active.devices` in the Class field.
2. Type `1000` in the Last Number field.
3. Type a short description for the number class.
   For example, type `Number of devices available`.
4. Type `true` in the Decrement field.
5. Type `0` in the Reset Point field.
6. Type `1` for the Increment/Decrement field value.
   Since the Decrement field is set to true, each number decreases by one.
7. Click **Add**.

The new sequential number record is added to the sequential number file.

Related topics

Sequential number file
Sequential number setup
Access the sequential number file
Create a simple number counter
Use prefix and suffix in sequential numbers
Update a sequential number record
Delete a sequential number record

1.2.8.5. Use prefix and suffix in sequential numbers

This example uses prefixes and suffixes to assign character type ID numbers to workstation devices. The format of the ID is: DEV<number>T where DEV is a fixed character prefix, <number > is a sequential number starting with 1, and T is a fixed character suffix.

1. For this example, type `devices` in the Class field.
2. Type `0` in the Last Number field.
3. Type a short description for the number class.
   For example, enter Workstation device ID counter.
4. Leave the Decrement field blank.
5. Type `1000` in the Reset Point field.
6. Type `1` for the Increment/Decrement field value.
7. Type `5` in the Length field.
8. Type `DEV` in the Prefix field.
9. Type `T` in the Suffix field.
10. Click **Add**.

The new sequential number record is added to the sequential number file. The first sequential number for the devices class is DEV00001T.

Related topics

Sequential number file
Sequential number setup
Access the sequential number file
Create a simple number counter
Use decrement in sequential numbers
Update a sequential number record
Delete a sequential number record

1.2.8.6. Update a sequential number record

Follow these steps:

1. Access the sequential number record. Use the search function or select the record from a record list.
2. Enter changes to the fields you want to update.
3. Click **Save**.

Related topics

Sequential number file
Sequential number setup
Access the sequential number file
Create a simple number counter
Use decrement in sequential numbers
Use prefix and suffix in sequential numbers
Delete a sequential number record

1.2.8.7. Delete a sequential number record

Follow these steps:

1. Access the sequential number record that you want to delete. Use the search function or select the record from a record list.
2. Click **Delete**.
3. Click **Yes** to confirm that you want to delete the record.

The form opens with the message: *Record deleted from the number file*.

Related concepts

Sequential number file
Sequential number setup
Access the sequential number file
Create a simple number counter
Use decrement in sequential numbers
Use prefix and suffix in sequential numbers
Update a sequential number record

© Copyright 2026 Open Text

For more info, visit https://docs.microfocus.com